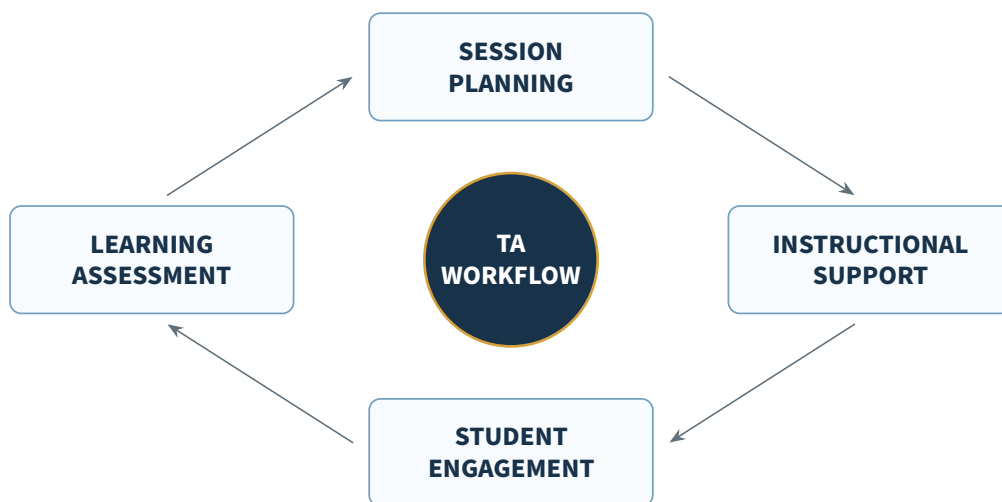


Artificial Intelligence

Teaching Assistant Guide



COURSE OUTLINE

Eight laboratory-guided modules covering search, fuzzy systems, robotics, natural language processing, and evolutionary optimization.

PREPARED BY

Reeyan Afzal Khan

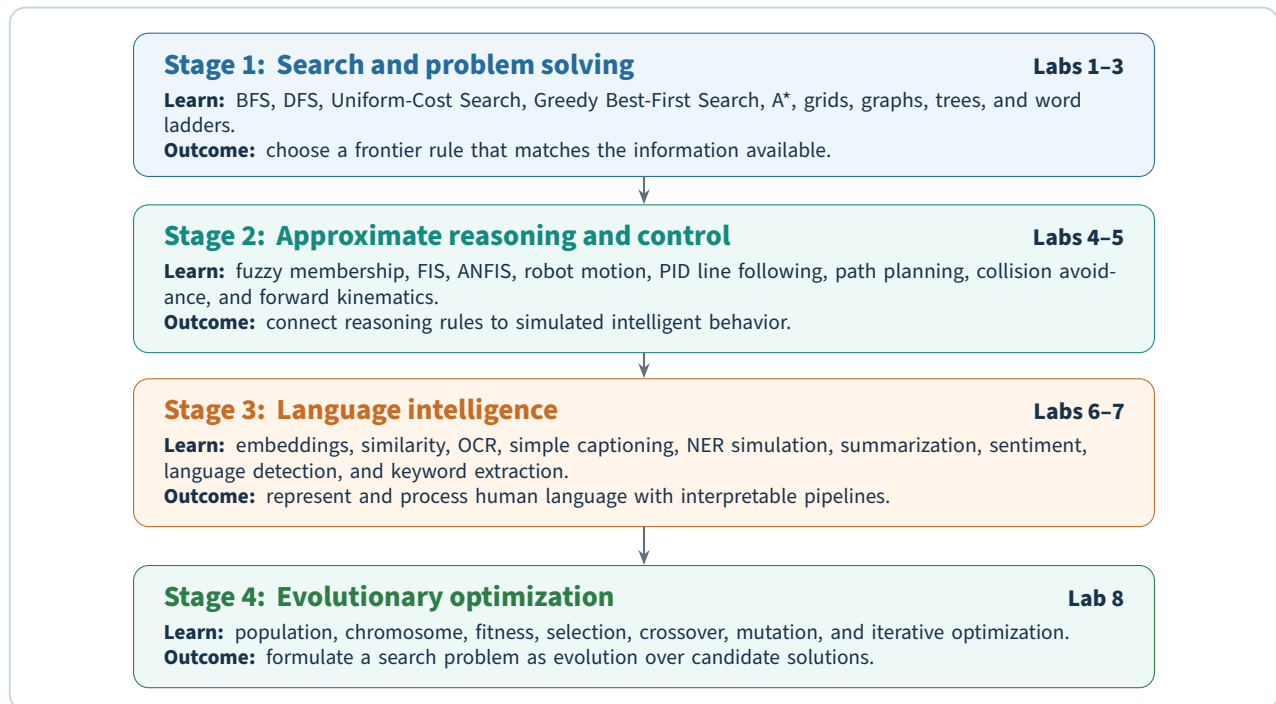
Graduate Research Assistant

Learning Path

Course Map: From Search to Intelligent Systems	ii
Stage 1 Search and Problem Solving	1
Chapter 1 Uninformed Search: BFS, DFS, and Uniform-Cost Search	2
Lab 1 of 8 — BFS, DFS, and Uniform-Cost Search	9
The Artificial-Intelligence Map for This Course	11
Chapter 2 Informed Search: Greedy Best-First Search and A*	12
Lab 2 of 8 — Greedy Best-First Search and A*	19
Chapter 3 Search Implementation Patterns	21
Lab 3 of 8 — Uninformed and informed search implementation	26
Stage 2 Approximate Reasoning and Intelligent Control	28
Chapter 4 Fuzzy Logic and ANFIS	29
Lab 4 of 8 — Fuzzy Logic and ANFIS in MATLAB	36
Chapter 5 Robotics Scenarios and Simulation	38
Lab 5 of 8 — Robotics scenarios and simulation in Python	47
Stage 3 Language Intelligence	49
Chapter 6 Natural Language Processing Foundations	50
Lab 6 of 8 — Natural Language Processing — Part A	56
Chapter 7 NLP Methods: Entities, Summaries, Sentiment, and Keywords	58
Lab 7 of 8 — Natural Language Processing — Part B	66
Stage 4 Evolutionary Optimization	68
Chapter 8 Genetic Algorithms	69
Lab 8 of 8 — Genetic Algorithm optimization	77
Chapter A Quick Reference	80

Course Map: From Search to Intelligent Systems

The eight laboratories introduce classical artificial intelligence (AI) through search, approximate reasoning, robotics, language processing, and evolutionary optimization. The map shows the progression and expected outcomes.



Key idea

The course does not use a single definition of “intelligence.” Instead, it studies several computational abilities: finding paths, using heuristic knowledge, reasoning with degrees of truth, controlling robots, processing language, and optimizing candidate solutions.

A repeatable reading routine

Before reading the code, identify:

1. the representation of the state, input, or candidate solution;
2. the rule that chooses the next state, action, or parent;
3. the stopping condition;
4. the returned or plotted result; and
5. one assumption under which that result is valid.

These questions provide a common framework across all chapters.

Stage 1

Search and Problem Solving

Uninformed Search: BFS, DFS, and Uniform-Cost Search

Chapter at a glance

Before you start: Be comfortable reading a graph, a grid, and a simple Python container such as a list, set, or dictionary.

By the end, you can: represent a search problem, explain frontier and visited sets, trace BFS and DFS, use Uniform-Cost Search on weighted graphs, and recognize when each method is appropriate.

The first laboratory introduces Breadth-First Search (BFS), Depth-First Search (DFS), and Uniform-Cost Search (UCS). These **uninformed** methods use no estimate of remaining distance; they differ in frontier order.

1.1 A search problem is more than a graph

The exercises use graphs, mazes, word transformations, and binary trees. These domains look different, but each can be expressed with the same ingredients.

Ingredient	Meaning in the laboratory problems
State	A graph node such as A; a maze coordinate; a word such as hot; or a tree node.
Start	The state where the search begins, for example A or S.
Goal	The desired destination, for example G or cog.
Successors	Legal next states, obtained from graph adjacency, grid movement, one-letter changes, or child links.
Path	The ordered sequence of states used to reach the current state.
Path cost	The total edge or movement cost accumulated so far; UCS explicitly uses this quantity.

Conceptual intuition

A search algorithm needs only a successor function and a rule for selecting the next frontier state; the state may represent a city, word, or maze cell.

1.2 The frontier and the visited set

Most graph-search implementations maintain two central structures.

Frontier States that have been discovered but not fully explored. The data structure used for this frontier gives BFS, DFS, and UCS their characteristic behavior.

Visited set States already processed. It prevents repeated work and is essential when the graph contains cycles or loops.

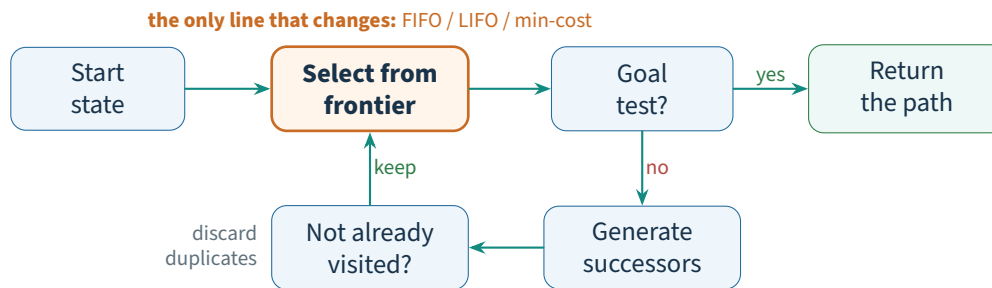


Figure 1.1. The graph-search loop shared by BFS, DFS, and UCS. Only the shaded box differs between the three algorithms: BFS takes the oldest frontier entry, DFS the newest, UCS the cheapest. Everything else — the successor generation, the goal test, the visited check — is identical code.

1.3 Breadth-First Search

Breadth-First Search expands states level by level. From A, it processes B and C before deeper states.

A first-in, first-out (FIFO) queue implements this rule:

- Step 1:** Put the start state in the queue.
- Step 2:** Remove the oldest state from the front.
- Step 3:** If it is the goal, return its path.
- Step 4:** Add unvisited successors to the back of the queue.
- Step 5:** Repeat until the queue is empty.

For the unweighted graph from Lab 1,

$$A \rightarrow \{B, C\}, \quad B \rightarrow \{D, E\}, \quad C \rightarrow \{F\},$$

with $D, E, F \rightarrow G$, BFS returns a path with the fewest edges because every step is treated as equal cost.

Worked example

With the stated neighbor order, BFS expands A, B, and C before the next layer; D then yields the path $A \rightarrow B \rightarrow D \rightarrow G$.

The queue trace below separates the state being expanded from the paths still waiting. The leftmost path is removed next.

expand	frontier after expansion
A	[A, B], [A, C]
B	[A, C], [A, B, D], [A, B, E]
C	[A, B, D], [A, B, E], [A, C, F]
D	[A, B, E], [A, C, F], [A, B, D, G]

The returned path has three edges and is selected by queue order, not a distance estimate.

1.3.1 BFS in a maze

The maze exercise uses four-direction movement. A cell is a state, and a legal successor is an in-bounds adjacent cell that is not a wall. BFS is suitable when every move has equal cost and the objective is the shortest number of steps.

```

from collections import deque

def bfs_grid(grid, start, goal):
    rows, cols = len(grid), len(grid[0])
    visited = {start}
    queue = deque([(start, [start])])

    while queue:
        (x, y), path = queue.popleft()
        if (x, y) == goal:
            return path

        for dx, dy in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
            nx, ny = x + dx, y + dy
            if (0 <= nx < rows and 0 <= ny < cols
                and grid[nx][ny] == 0
                and (nx, ny) not in visited):
                visited.add((nx, ny))
                queue.append(((nx, ny), path + [(nx, ny)]))

    return None

```

Figure 1.2 shows why this works. BFS reaches every cell at distance 1 before any at distance 2, so the first time it touches the goal it has arrived by a shortest route.



Each number is the step at which BFS first reaches that cell.

The goal is found at distance 8, and no shorter route exists: BFS could not have reached it earlier without skipping a ring.

Dark cells are walls and are never entered.

Figure 1.2. BFS in a grid, shaded by distance from the start. The frontier is always one connected ring, so the goal is first touched at its true shortest distance. Dark cells are walls.

Common mistake

Mark a state visited when it is inserted into the BFS queue, not only when it is removed. Otherwise the same state can be inserted many times by different parents.

1.4 Depth-First Search

Depth-First Search follows one branch as far as possible before trying alternatives. A stack or recursion naturally implements this behavior.

Step 1: Start at the current state and perform the goal test.

Step 2: Mark the state visited.

Step 3: Choose an unvisited successor.

Step 4: Recurse down that branch.

Step 5: If the branch fails, return to the previous state and try the next successor.

The maze implementation expresses this behavior directly through recursion:

```

def dfs_maze(grid, x, y, path, visited, goal):
    if (x, y) == goal:

```

```

return path

visited.add((x, y))
for dx, dy in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
    nx, ny = x + dx, y + dy
    if (0 <= nx < len(grid) and 0 <= ny < len(grid[0])
        and grid[nx][ny] == 0
        and (nx, ny) not in visited):
        result = dfs_maze(
            grid, nx, ny,
            path + [(nx, ny)],
            visited, goal
        )
    if result:
        return result
return None

```

Conceptual intuition

BFS expands a wavefront; DFS follows one branch. DFS does not guarantee a minimum-edge path.

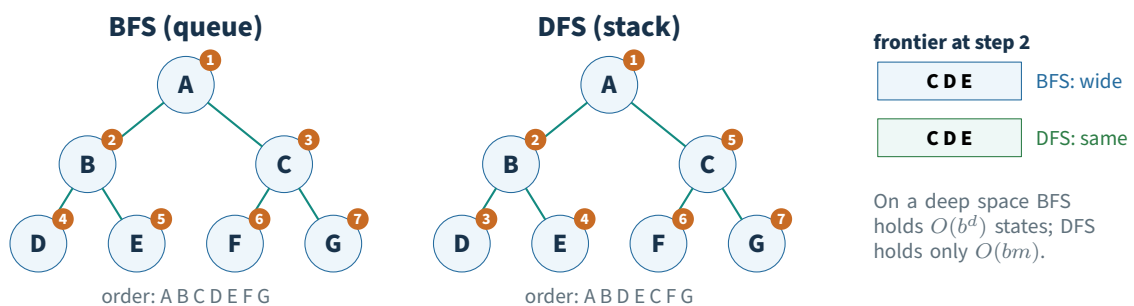


Figure 1.3. BFS and DFS on one tree, each node numbered by visit order. BFS finishes a level before descending; DFS exhausts B's subtree before touching C. The frontier column is the memory argument: wide for BFS, shallow for DFS.

With left children first, BFS completes each depth while DFS completes the left branch first. Neighbor order changes the sequence, not this distinction.

1.4.1 DFS preorder on a binary tree

The Lab 1 tree has root 1, children 2 and 3, and leaves 4–7. Preorder DFS visits the current node before its left and right subtrees:

1, 2, 4, 5, 3, 6, 7.

```

def dfs_preorder(root):
    if not root:
        return []
    return [root.val] + dfs_preorder(root.left) + dfs_preorder(root.right)

```

1.5 Uniform-Cost Search

Uniform-Cost Search expands the frontier path with the smallest accumulated cost $g(n)$, using a min-priority queue instead of BFS's FIFO queue.

UCS assumes nonnegative edge costs; negative costs invalidate its usual visited-set logic.

For the weighted Lab 1 graph,

$$\begin{array}{lll}
 A \rightarrow B : 1, & A \rightarrow C : 4, & \\
 B \rightarrow D : 3, & B \rightarrow E : 2, & C \rightarrow F : 5, \\
 D \rightarrow G : 4, & E \rightarrow G : 1, & F \rightarrow G : 2.
 \end{array}$$

The path through B and E has cost

$$1 + 2 + 1 = 4,$$

which is lower than the alternatives shown in the handout.

Worked example

UCS starts with $(0, A)$. Expanding A inserts $(1, B)$ and $(4, C)$; expanding B inserts $(4, D)$ and $(3, E)$. It therefore selects E before C , inserts $(4, G)$, and returns $A \rightarrow B \rightarrow E \rightarrow G$ with optimal cost 4. Equal-cost tie order does not affect that minimum cost.

```

import heapq

def ucs(graph, start, goal):
    pq = [(0, [start])]
    visited = set()

    while pq:
        cost, path = heapq.heappop(pq)
        node = path[-1]
        if node == goal:
            return path, cost

        if node not in visited:
            visited.add(node)
            for neighbor, weight in graph[node]:
                heapq.heappush(
                    pq,
                    (cost + weight, path + [neighbor])
                )

    return None

```

Figure 1.4 puts the two answers side by side. Both paths use three edges, so BFS sees no reason to prefer either; UCS adds the weights and finds one costs twice the other. That is what “BFS is optimal only for equal step costs” means in practice.

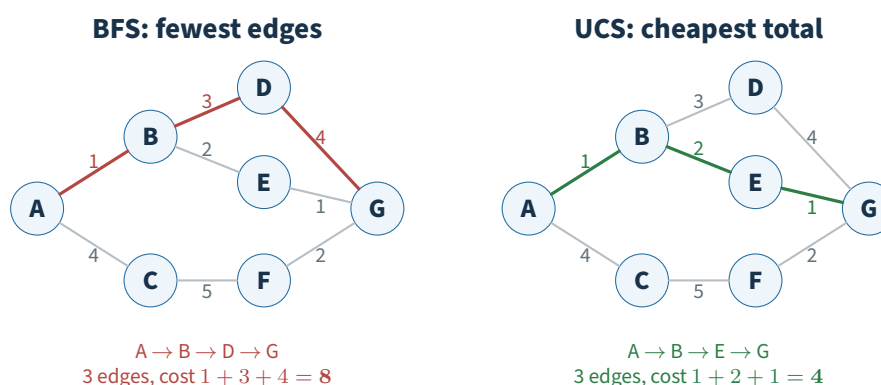


Figure 1.4. The same weighted graph solved twice. BFS returns the first three-edge path it encounters, costing 8. UCS orders its frontier by accumulated cost $g(n)$ and returns the three-edge path costing 4. Neither algorithm is wrong; they are answering different questions.

Key idea

BFS orders the frontier by depth, DFS by recency/depth, and UCS by accumulated path cost. The graph representation may stay the same while only the frontier policy changes.

1.6 Comparing the three methods

Method	Frontier rule	Main structure	What the lab examples emphasize
BFS	Shallowest first	FIFO queue	Shortest path in equal-cost graph/grid; level-order tree traversal
DFS	Deepest first	Stack / recursion	Reachability, maze exploration, preorder tree traversal
UCS	Lowest $g(n)$	Min-priority queue	Least-cost path in a weighted graph

1.6.1 The four properties every search algorithm is judged on

Choosing between these methods means asking four standard questions, not just “which one found a path”.

Completeness If a solution exists, is the algorithm guaranteed to find it?

Optimality Is the solution it returns guaranteed to be the cheapest one?

Time complexity How many states might it expand?

Space complexity How many states might it hold in memory at once?

Write b for the **branching factor** (successors per state), d for the depth of the shallowest goal, and m for the maximum depth of the search space.

Method	Complete?	Optimal?	Time	Space
BFS	Yes, if b is finite	Yes, but only when every step costs the same	$O(b^d)$	$O(b^d)$
DFS	Not in general	No	$O(b^m)$	$O(bm)$
UCS	Yes, if step costs $\geq \epsilon > 0$	Yes, for any non-negative costs	$O(b^{1+\lceil C^*/\epsilon \rceil})$	same as time

Here C^* is the cost of the optimal solution and ϵ the smallest step cost.

Figure 1.5 shows what $O(b^d)$ costs in practice. At branching factor 3, a goal eight levels down puts about 9 800 states in memory; at branching factor 5 the same depth needs nearly 500 000. This is why the space column, not the time column, decides which algorithm survives.

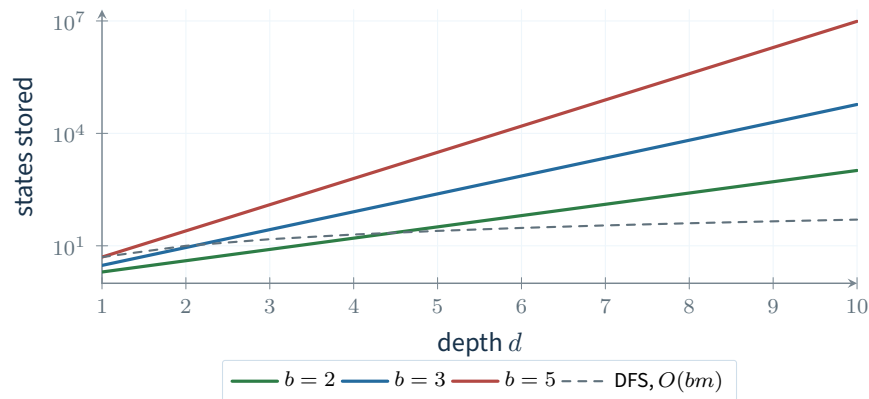


Figure 1.5. States a BFS frontier may hold at depth d , for three branching factors. The vertical axis is logarithmic: each straight line is exponential growth. DFS stores $O(bm)$ instead, which on these scales is a flat line near the bottom.

Key idea

Read the space column first. BFS and UCS store the whole frontier, which grows exponentially — memory, not time, is what usually stops them on a real problem. DFS needs only the current branch, $O(bm)$, which is why it survives on deep spaces that exhaust BFS. That is the trade: DFS buys its small memory footprint by giving up both completeness and optimality.

Common mistake

Two entries above are easy to misread.

“BFS is optimal” holds only for *equal* step costs. On the weighted Lab 1 graph, BFS returns $A \rightarrow B \rightarrow D \rightarrow G$ costing 8 while UCS returns $A \rightarrow B \rightarrow E \rightarrow G$ costing 4 — both use three edges. Fewest edges and cheapest are different questions, and BFS only answers the first.

“DFS is not complete” refers to spaces that are infinite or contain cycles: DFS can descend forever and never come back. On a *finite* graph with a visited set — every graph in this lab — DFS does terminate and will find a goal if one is reachable. It still gives no guarantee about which one.

PRACTICAL MILESTONE

Lab 1 of 8

BFS, DFS, and Uniform-Cost Search

Search foundations across graphs, mazes, words, and trees

Coverage. Chapter 1 — Breadth-First, Depth-First, and Uniform-Cost Search.

Source files. Lab01/search_uninformed.py — run it with `py search_uninformed.py`.

Goal. Use the same search vocabulary across four representations and explain how the frontier rule changes behavior.

Scaffolding

Use the supplied problem instances and the reusable implementations in this chapter. Trace each case before running it.

Part A — BFS problem set.

1. Find a path from A to G in the unweighted interpretation of the supplied graph.
2. Find a path from S (start) to G (goal) in the supplied two-dimensional (2D) maze, where “.” is open and “#” is a wall.
3. Transform `hit` to `cog` using the valid word list `hit`, `hot`, `dot`, `dog`, `lot`, `log`, `cog`.
4. Traverse the seven-node binary tree level by level.

Part B — DFS problem set.

Repeat the graph, maze, word-ladder, and tree tasks using DFS. For the tree, use preorder traversal.

Part C — UCS problem set.

Use the weighted graph from the handout, then solve the weighted grid and the graph with loops. Record both the returned path and its total cost.

Practice

Record the frontier-removal order, then change one neighbor ordering and compare the returned path.

Lab deliverable

Submit BFS, DFS, and UCS implementations; successful runs for each representation; and a brief comparison of frontier order, path length, and cost.

Check your understanding

1. Why does BFS ignore the numerical edge weights in the first graph problem?
2. What changes in the data structure when moving from BFS to UCS?
3. Why is a visited set especially important for the “graph with loops” UCS problem?
4. What traversal order does DFS preorder produce for the seven-node binary tree in Lab 1?

Quick-check answers

1. BFS treats every edge as one step, so numerical weights are outside its ordering rule.
2. Replace the FIFO queue with a min-priority queue ordered by cumulative cost g .
3. Without a visited or best-cost record, loop edges continually create longer cyclic paths.
4. 1, 2, 4, 5, 3, 6, 7.

Chapter recap

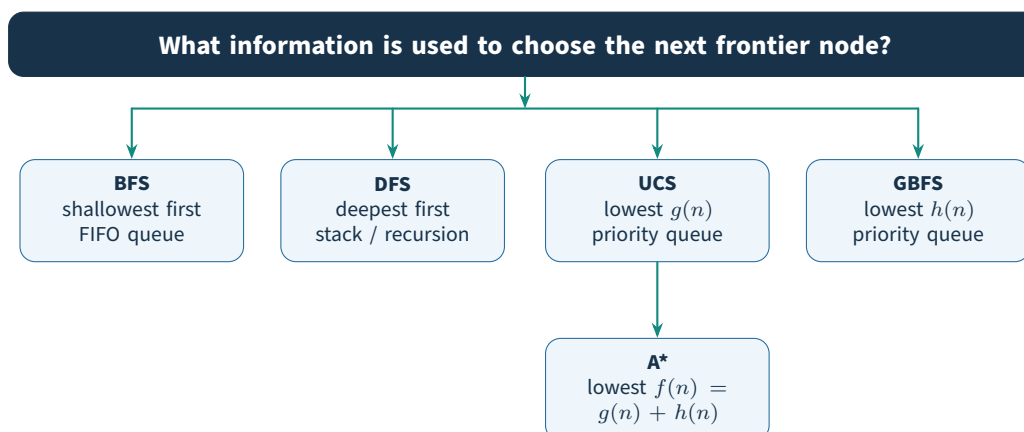
- Search problems specify states, successors, a start, a goal test, and optionally path costs.
- BFS uses a FIFO queue and is suited to equal-cost shortest-path problems.
- DFS follows one branch before backtracking, using recursion or a stack.
- UCS uses a priority queue ordered by cumulative cost.

The Artificial-Intelligence Map for This Course

Select an AI method by identifying the available information and required output, rather than asking which algorithm is universally best.

Problem signal	Main techniques in the labs	Typical output
Connectivity only	BFS, DFS	A reachable path or traversal order
Edge costs	Uniform-Cost Search	A least-cost path
Heuristic estimate	Greedy Best-First Search	A goal-directed path
Cost + heuristic	A* Search	A path balancing known cost and estimated remaining cost
Imprecise linguistic rules	Fuzzy Logic, FIS	A crisp control value from fuzzy rules
Input-output examples	ANFIS	A learned Sugeno-type fuzzy model
Robot state and feedback	Kinematics, PID, A* planning	Motion, path, or control response
Text or images containing text	Embeddings, OCR, simple NLP rules	Similarity, labels, summaries, translations, keywords
Candidate solution population	Genetic Algorithm	An optimized or near-optimized solution

Search methods at a glance



Key idea

BFS and DFS differ by frontier order. UCS uses path cost, GBFS uses only a heuristic, and A* combines both.

Informed Search: Greedy Best-First Search and A*

Chapter at a glance

Before you start: Understand BFS/DFS/UCS and the idea of a priority queue.

By the end, you can: define a heuristic, trace Greedy Best-First Search, compute $g(n)$, $h(n)$, and $f(n)$, explain A*, and apply Manhattan distance to grid navigation.

Lab 2 introduces a **heuristic**: an estimate of remaining distance or cost used to guide the frontier.

2.1 Heuristic search

Let $h(n)$ denote the heuristic value of node n . Smaller values in the laboratory examples mean “looks closer to the goal.” The goal itself has $h(G) = 0$.

Quantity	Meaning	Used by
$g(n)$	Actual path cost accumulated from the start to node n	UCS and A*
$h(n)$	Estimated remaining cost or distance from n to the goal	GBFS and A*
$f(n)$	$g(n) + h(n)$, an estimated total cost through n	A*

Key idea

Greedy Best-First Search asks, “Which frontier state appears closest to the goal?” A* asks, “Which frontier state gives the best combination of cost already paid and estimated cost still remaining?”

Figure 2.1 shows the three quantities on a single node. The distinction is entirely about which part of the journey each algorithm is willing to look at: UCS looks only backwards, GBFS only forwards, and A* at both.

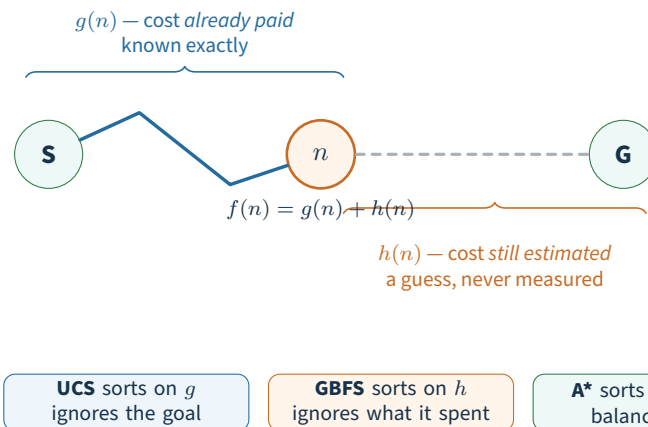


Figure 2.1. The decomposition $f(n) = g(n) + h(n)$. The cost g behind the node is known exactly; the estimate h ahead of it is a guess. Each algorithm is defined by which half it sorts the frontier on.

2.2 Greedy Best-First Search

Greedy Best-First Search (GBFS) orders the frontier by heuristic value only. It ignores the cost already spent reaching a node.

The Lab 2 graph assigns

$$h(A) = 6, h(B) = 4, h(C) = 5, h(D) = 2, h(E) = 3, h(F) = 2, h(G) = 0.$$

From A, the handout chooses B because $h(B) = 4 < h(C) = 5$. From B, it prefers D because $h(D) = 2 < h(E) = 3$. Since D is a dead end, the search later explores another frontier state and eventually reaches G through E.

```
import heapq

def greedy_best_first(graph, heuristic, start, goal):
    pq = [(heuristic[start], [start])]
    visited = set()

    while pq:
        _, path = heapq.heappop(pq)
        node = path[-1]
        if node == goal:
            return path

        if node in visited:
            continue
        visited.add(node)

        for neighbor in graph[node]:
            if neighbor not in visited:
                heapq.heappush(
                    pq,
                    (heuristic[neighbor], path + [neighbor])
                )

    return None
```

Common mistake

GBFS may prefer a low-heuristic state reached by an expensive path because it ranks only $h(n)$.

2.3 Manhattan distance in a grid

The choice of distance formula must match the moves the robot can actually make (Figure 2.2). With four-direction movement, Manhattan distance is the exact number of steps needed on an empty grid, so it is admissible and tight. Euclidean distance cuts the corner and always *under-estimates*, which is still admissible but weaker: it guides the search less and A* expands more cells.

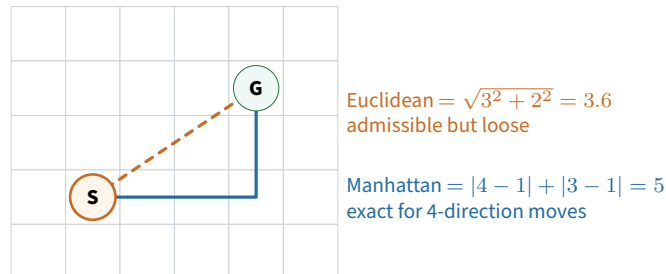


Figure 2.2. Two estimates from (1, 1) to (4, 3). With four-direction movement the true cost is 5 steps, which Manhattan reports exactly. Euclidean reports 3.6: admissible, but a looser guide, so A* expands more cells.

The Lab 2 robot-maze problem uses the Manhattan-distance heuristic. For two grid coordinates $a = (x_a, y_a)$ and $b = (x_b, y_b)$,

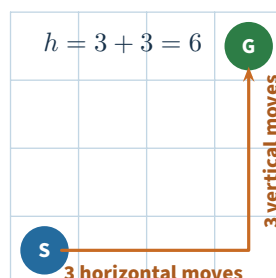
$$h(a, b) = |x_a - x_b| + |y_a - y_b|.$$

It measures how many horizontal and vertical coordinate differences remain when diagonal movement is not used.

```
def manhattan(a, b):
    return abs(a[0] - b[0]) + abs(a[1] - b[1])
```

Worked example

For start (0, 0) and goal (3, 3), the start heuristic is 6. A move to (0, 1) reduces the heuristic to 5. GBFS repeatedly prefers frontier positions with the smallest remaining Manhattan distance, subject to walls and the positions already explored.



Obstacles may lengthen the route, but the six coordinate changes remain necessary; Manhattan distance is therefore a lower bound for unit-cost four-direction movement.

2.4 Word-ladder heuristic

For the Lab 2 word ladder, the heuristic is the number of letters that differ from the goal word. If the goal is cog, then dog differs in one position while dot differs in two positions.

```
def word_h(word, goal):
    return sum(a != b for a, b in zip(word, goal))
```

The representation changes from coordinates to strings, but $h(n)$ retains the same role.

2.5 A* Search

A* Search combines actual and estimated cost:

$$f(n) = g(n) + h(n).$$

Here, $g(n)$ penalizes costly paths while $h(n)$ guides the search toward the goal.

Worked example

Suppose X has $(g, h) = (5, 3)$ and Y has $(1, 5)$. GBFS chooses X because $3 < 5$; A* chooses Y because $f(Y) = 6 < f(X) = 8$.

For A* to guarantee an optimal result, the heuristic must not overestimate the cheapest remaining cost. Such a heuristic is **admissible**. A commonly used stronger condition is **consistency**: for every edge $n \rightarrow n'$ of cost c ,

$$h(n) \leq c + h(n').$$

Manhattan distance is consistent for four-direction movement when every move costs at least one. If moves can cost less than one, scale the heuristic by the minimum possible move cost.

For the supplied graph:

$$\begin{array}{ll} A \rightarrow B : 1, & A \rightarrow C : 4, \\ B \rightarrow D : 3, & B \rightarrow E : 1, \\ E \rightarrow G : 2, & C \rightarrow F : 5, \quad F \rightarrow G : 1. \end{array}$$

with the handout's heuristic

$$h(A) = 7, h(B) = 6, h(C) = 4, h(D) = 3, h(E) = 2, h(F) = 2, h(G) = 0.$$

The optimal route is

$$A \rightarrow B \rightarrow E \rightarrow G$$

with path cost $1 + 1 + 2 = 4$.

2.5.1 Is that heuristic admissible? Check, do not assume

The condition above is worth applying to the handout's own numbers rather than reading past it. Let $h^*(n)$ be the *true* cheapest cost from n to the goal, obtained by searching backwards from G . Admissibility requires $h(n) \leq h^*(n)$ at every node.

Node	$h(n)$	true $h^*(n)$	Admissible?
A	7	4	no — overestimates by 3
B	6	3	no — overestimates by 3
C	4	6	yes
D	3	—	G is unreachable from D
E	2	2	yes (exact)
F	2	1	no — overestimates by 1
G	0	0	yes

Figure 2.3 plots the table. Any bar rising above the diagonal is a node where the estimate exceeds the truth, and a single such node is enough to void the guarantee.

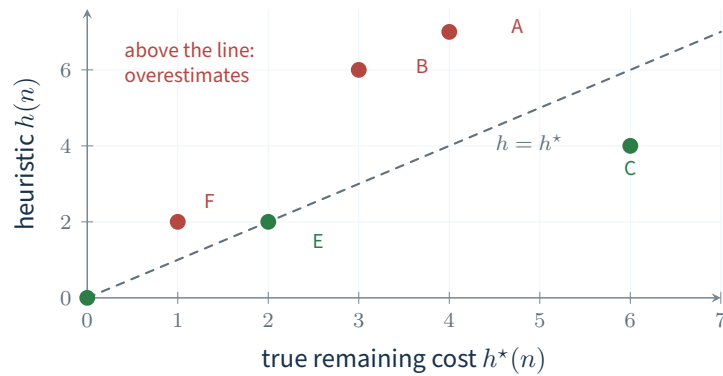


Figure 2.3. The handout's heuristic against the true remaining cost. Admissibility requires every point to lie on or below the dashed line; A, B, and F sit above it. Halving the heuristic — the largest factor the worst node allows — pulls all of them under.

Common mistake

This heuristic is **not admissible**, so A* carries no optimality guarantee on this graph. It still returns A → B → E → G at cost 4, which happens to be optimal — but that is luck, not proof, and a differently shaped graph with the same style of heuristic can return a worse path. Report which heuristics in the lab are admissible and which are not. “A* found it, so it must be optimal” is exactly the reasoning the admissibility condition exists to prevent.

Practice

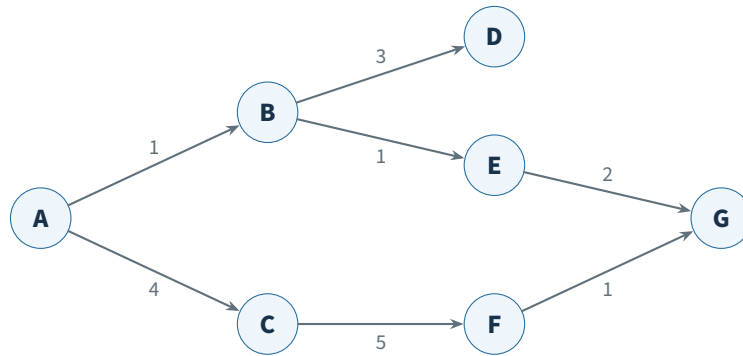
Lab02/search_informed.py computes the h^* column by running a backwards search from the goal and prints this table, so you can verify the claim instead of taking it on trust. Then repair the heuristic by scaling every value by a constant k . Admissibility needs $k h(n) \leq h^*(n)$ at every node, so k is limited by the worst node:

$$k \leq \min_n \frac{h^*(n)}{h(n)} = \min \left(\frac{4}{7}, \frac{3}{6}, \frac{6}{4}, \frac{2}{2}, \frac{1}{2} \right) = \frac{1}{2},$$

with B and F both binding. Halving the heuristic makes the whole table read yes. Scaling by $4/7$ — the factor that fixes the largest single error, at A — is not enough, and checking that for yourself is the point of the exercise.

Conceptual intuition

An inadmissible heuristic makes A* behave more like Greedy Best-First Search: it trusts the estimate too much and can commit to a branch before the cost evidence says it should. Admissibility is what keeps the g term in charge when the two disagree.



2.5.2 A* implementation pattern

The Lab 3 reference implementation stores both g and f in the heap:

```

def astar_graph(graph, heuristic, start, goal):
    import heapq
    pq = [(heuristic[start], 0, [start])]
    best_g = {start: 0}

    while pq:
        f, g, path = heapq.heappop(pq)
        node = path[-1]
        if g > best_g.get(node, float('inf')):
            continue  # stale, more expensive entry
        if node == goal:
            return path, g

        for neighbor, cost in graph[node]:
            new_g = g + cost
            if new_g < best_g.get(neighbor, float('inf')):
                best_g[neighbor] = new_g
                heapq.heappush(
                    pq,
                    (new_g + heuristic[neighbor],
                     new_g,
                     path + [neighbor])
                )

    return None

```

The `best_g` dictionary replaces costlier routes and identifies stale queue entries.

2.6 A* in a weighted grid

Lab 2 supplies a 5×5 grid in which each cell contains a movement cost and asks for a path from $(0, 0)$ to $(4, 4)$. The search cost g accumulates grid-cell costs, while Manhattan distance supplies h .

```

def astar_grid(grid, start, goal):
    import heapq

    def h(a, b):
        return abs(a[0] - b[0]) + abs(a[1] - b[1])

    rows, cols = len(grid), len(grid[0])
    pq = [(h(start, goal), 0, [start])]
    best_g = {start: 0}

    while pq:
        f, g, path = heapq.heappop(pq)
        x, y = path[-1]
        if g > best_g.get((x, y), float('inf')):
            continue
        if (x, y) == goal:
            return path, g

```

```

for dx, dy in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
    nx, ny = x + dx, y + dy
    if 0 <= nx < rows and 0 <= ny < cols:
        new_g = g + grid[nx][ny]
        if new_g < best_g.get((nx, ny), float('inf')):
            best_g[(nx, ny)] = new_g
            heapq.heappush(
                pq,
                (new_g + h((nx, ny), goal),
                 new_g,
                 path + [(nx, ny)])
            )
return None

```

2.7 UCS, GBFS, and A* side by side

Method	Priority	Knowledge used	Main behavior in the lab material
UCS	$g(n)$	Edge/path costs	Favors the cheapest path discovered so far
GBFS	$h(n)$	Heuristic only	Moves toward states that look closest to the goal
A*	$g(n) + h(n)$	Cost and heuristic	Balances cost-so-far with estimated remaining cost

The table states the rule; Figure 2.4 shows the consequence. All three find a path, but they examine very different numbers of cells doing it.

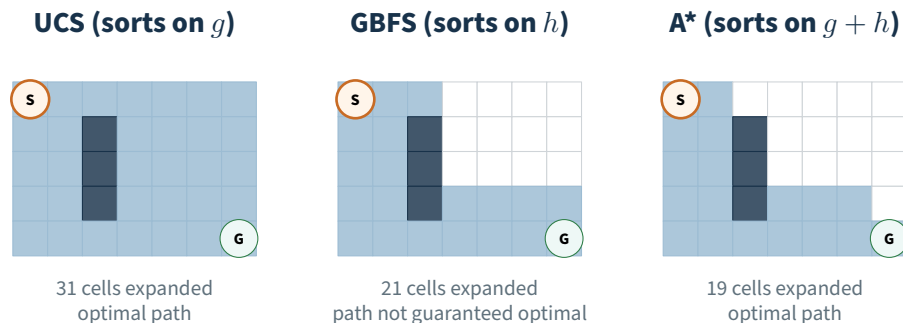


Figure 2.4. Cells expanded before the goal is reached, on an identical grid. UCS spreads in all directions because it cannot see the goal. GBFS runs almost straight at it but is drawn into the dead end. A* expands a narrow corridor and still returns the optimal path.

Key idea

Shaded cells are wasted work. UCS wastes it exploring away from the goal; GBFS wastes less but forfeits optimality. Only A* spends little *and* promises the cheapest path — and only while the heuristic stays admissible. That is why the admissibility check earlier is not a formality.

PRACTICAL MILESTONE

Lab 2 of 8

Greedy Best-First Search and A*

Heuristic guidance on graphs, grids, cities, and words

Coverage. Chapter 2 — Greedy Best-First and A*.

Source files. Lab02/search_informed.py — run it with `python search_informed.py`.

Goal. Build and inspect priority queues whose ordering is controlled by $h(n)$ or $g(n) + h(n)$.

Scaffolding

Keep the supplied heuristics unchanged while tracing each priority-queue decision.

Part A — GBFS.

Implement and test the four handout problems:

1. Basic graph from A to G using the supplied heuristic values.
2. Maze/grid navigation guided by Manhattan distance.
3. City-navigation graph beginning at S with the supplied heuristic dictionary.
4. Word ladder from `hit` to `cog` using letter mismatch as $h(w)$.

Part B — A*.

Use the weighted graph and heuristic dictionary from the handout to recover the reported route A → B → E → G with total cost 4. Then use A* on the weighted 5 × 5 grid from (0, 0) to (4, 4).

Practice

Print g , h , and f for each A* removal and h for GBFS. Explain one ranking difference.

Common mistake

Check the supplied heuristic before trusting A*'s answer. A* is only guaranteed to return an optimal path when h is *admissible* — never larger than the true remaining cost h^* . The hand-out's Part B heuristic is not:

Node	$h(n)$	true $h^*(n)$	admissible?
A	7	4	no
B	6	3	no
F	2	1	no

The search still returns A → B → E → G at cost 4, which happens to be optimal — but that is luck, not the guarantee. `search_informed.py` computes h^* by searching backwards from the goal and prints this table, so you can verify the claim rather than accept it. Report which heuristics in the lab are admissible and which are not.

Lab deliverable

Submit both implementations, all six cases, and a table comparing UCS, GBFS, and A* priorities.

Check your understanding

1. What information does GBFS ignore that A* uses?
2. For a grid state (2, 1) and goal (4, 4), what is the Manhattan heuristic?
3. In the Lab 2 A* graph, what is the total cost of A → B → E → G?
4. Why can two different domains use different heuristic formulas while using the same GBFS or A* control structure?

Quick-check answers

1. GBFS ignores the accumulated path cost $g(n)$.
2. $|2 - 4| + |1 - 4| = 5$.
3. $1 + 1 + 2 = 4$.
4. The control loop only needs a numerical estimate; each domain defines that estimate from its own representation.

Chapter recap

- A heuristic $h(n)$ estimates remaining distance or cost to the goal.
- GBFS orders the frontier by $h(n)$ and does not include path cost already incurred.
- A* orders the frontier by $f(n) = g(n) + h(n)$.
- Manhattan distance guides grids; letter mismatch guides word ladders.

Search Implementation Patterns

Chapter at a glance

Before you start: Know the conceptual difference among BFS, DFS, UCS, GBFS, and A*.

By the end, you can: recognize reusable Python patterns for search, convert graphs to grids or trees without changing the search logic, and debug common frontier/visited mistakes.

Lab 3 consolidates the search methods around five elements: frontier, visited set, successor loop, goal test, and path representation.

3.1 The generic graph-search loop

The methods share the following control flow.

```
frontier = make_frontier(start)
visited = set()

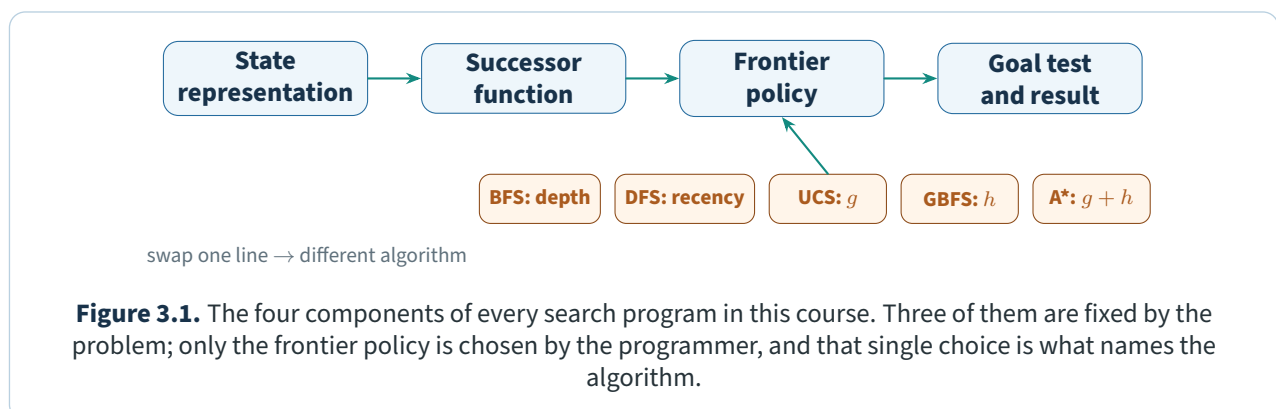
while frontier:
    state, path, cost = remove_next(frontier)

    if state == goal:
        return path, cost

    if state in visited:
        continue
    visited.add(state)

    for successor in successors(state):
        add_to_frontier(frontier, successor, path, cost)
```

What changes is `remove_next` and the priority attached to each state.

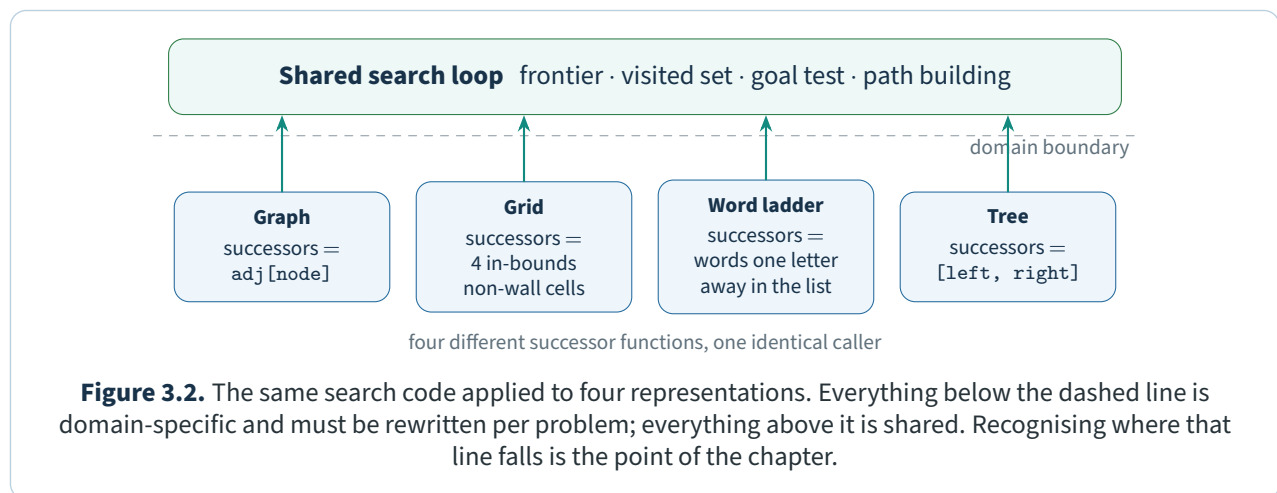


The domain defines states and successors; the algorithm defines frontier policy. The same control loop can therefore serve cities, grids, or words.

Algorithm	Frontier implementation	Information stored or prioritized
BFS	<code>collections.deque</code>	Path; oldest discovered state first
DFS	Recursion or list stack	Current recursive path / most recent state
UCS	heapq	$(g, path)$
GBFS	heapq	$(h, path)$
A*	heapq	$(f, g, path)$

3.2 Representation changes; search structure remains

Figure 3.2 makes the claim concrete across the four domains used in Labs 1–3. The successor function looks completely different in each — adjacency lookup, coordinate arithmetic, letter substitution, child pointers — but it returns the same thing every time: a list of states. Above that line, the search code never changes.



3.2.1 Graphs

A graph dictionary directly maps a node to its neighbors. Weighted graphs map to (neighbor, cost) pairs.

```
graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('D', 3), ('E', 2)],
    'C': [('F', 5)],
    'D': [('G', 4)],
    'E': [('G', 1)],
    'F': [('G', 2)],
    'G': []
}
```

3.2.2 Grids

A grid does not list neighbors explicitly. The successor function computes adjacent coordinates and checks bounds and cell conditions.

```
for dx, dy in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
    nx, ny = x + dx, y + dy
    if 0 <= nx < rows and 0 <= ny < cols:
        # apply the problem's wall or cost rule here
        ...
```

3.2.3 Word ladders

The Lab 3 GBFS example stores valid one-letter transitions explicitly as a graph. The heuristic compares the current word with the goal.

```
def greedy_word_ladder(words, start, goal):
    import heapq

    def h(w):
        return sum(1 for a, b in zip(w, goal) if a != b)

    pq = [(h(start), [start])]
    visited = set()

    while pq:
        _, path = heapq.heappop(pq)
        current = path[-1]
        if current == goal:
            return path

        if current in visited:
            continue
        visited.add(current)
        for neighbor in words.get(current, []):
            if neighbor not in visited:
                heapq.heappush(pq, (h(neighbor), path + [neighbor]))
    return None
```

3.2.4 Trees

Tree traversal uses child references instead of a graph dictionary. BFS enqueues children; DFS recursively visits them.

```
def bfs_tree(root):
    from collections import deque
    if not root:
        return []

    queue = deque([root])
    result = []
    while queue:
        node = queue.popleft()
        result.append(node.val)
        if node.left:
            queue.append(node.left)
        if node.right:
            queue.append(node.right)
    return result
```

3.3 Path storage as a teaching device

The teaching implementations store the full path in each frontier entry:

```
queue.append((nx, ny), path + [(nx, ny)])
```

Conceptual intuition

Path copying simplifies traces and permits immediate path return, although larger systems often store parent links to reduce memory use.

3.4 Visited-state discipline

The examples update visited sets at slightly different points. The purpose is always the same: avoid processing a state repeatedly.

Debugging task

Take the “graph with loops” from Lab 1, which contains edges back from B and C to A. Remove the visited set from UCS and print the first 30 popped paths. Explain why the priority queue keeps receiving cyclic paths even though a goal exists.

UCS may still reach a cheap goal, but cyclic paths such as A → B → A → B inflate the queue. Record popped states and maximum frontier size to expose the defect.

3.5 Priority queues and tuple ordering

Python’s `heapq` removes the smallest tuple lexicographically. The laboratory code intentionally places the numerical priority first:

```
# UCS
heapq.heappush(pq, (cost + weight, path + [neighbor]))

# GBFS
heapq.heappush(pq, (heuristic((nx, ny), goal), path + [(nx, ny)]))

# A*
heapq.heappush(pq, (new_g + heuristic((nx, ny), goal),
                  new_g,
                  path + [(nx, ny)]))
```

The first tuple element sets priority; later fields carry cost or path data. Figure 3.3 shows what `heapq` actually compares.

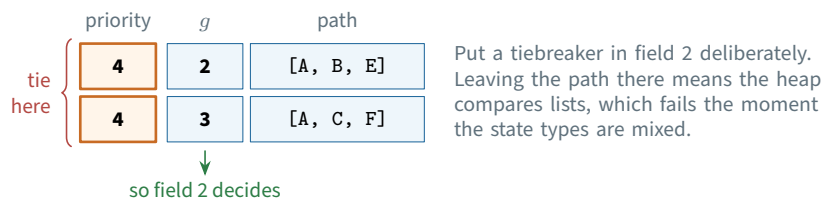


Figure 3.3. Tuples are compared left to right. Field 1 is the priority; when two entries tie there, Python silently moves to field 2 — the path list in this code. That works only while all states are mutually comparable.

Common mistake

When two priorities tie, Python moves on to the *next* tuple element — and in the code above that is the path list. Comparing two lists compares their elements, so this only works while every state is mutually comparable.

The lab states are homogeneous (all strings, or all coordinate tuples), so the ties resolve quietly and the code runs. Mix the types and the same line raises at run time:

```
heapq.heappush(pq, (4, ['A', 1]))
heapq.heappush(pq, (4, ['A', 'B']))
# TypeError: '<' not supported between instances of 'str' and 'int'
```

It is also a correctness smell even when it works: the tie is broken by whichever path happens to sort first, so the algorithm’s output depends on state names rather than on the search.

The fix is one unique increasing integer between the priority and the payload, which makes ties

resolve by insertion order and stops the payload from ever being compared:

```
counter = 0
...
counter += 1
heapq.heappush(pq, (cost + weight, counter, path + [neighbor]))
```

3.6 A single comparison harness

Run all compatible algorithms through one reporting function:

```
def show_result(name, result):
    print(f"\n{name}")
    if result is None:
        print("No path found")
    else:
        print(result)
```

Compare two questions:

1. Did the algorithm find a goal?
2. What rule caused it to choose this particular path or cost?

PRACTICAL MILESTONE

Lab 3 of 8

Uninformed and informed search implementation

One Python search toolkit

Coverage. Chapter 3 — one reusable search skeleton behind all five strategies.

Source files. Lab03/search_toolkit.py (the module, with a self-test: `py search_toolkit.py`) and Lab03/run_all.py (the driver that prints the comparison table).

Goal. Turn the repeated code into a small, testable search toolkit.

Scaffolding

Use the supplied reference functions to build one consistent toolkit; the weighted maze reuses the A* grid pattern.

Part A — Uninformed functions.

Place the following in one module: `bfs_grid`, `dfs_maze`, `ucs`, `bfs_tree`, and `dfs_preorder`.

Part B — Informed functions.

Add `greedy_bfs`, `astar_grid`, `astar_graph`, and `greedy_word_ladder`. Use the weighted-grid example to confirm that the A* grid implementation can accumulate cell costs.

Part C — Cross-algorithm trace.

Choose one graph and instrument each compatible algorithm to print the frontier priority, popped state, path, and visited-set size.

Practice

Summarize actual outputs by algorithm, data structure, priority, path, and cost.

Lab deliverable

Submit the search module plus a driver notebook/script that runs every Lab 3 example and prints a compact comparison table.

Check your understanding

1. Which one line changes conceptually when you move from UCS to GBFS: successor generation or priority calculation?
2. Why can the same Manhattan heuristic function be reused in GBFS and A*?
3. What is the purpose of carrying g separately from f in the A* heap entry?
4. In a tree, what plays the role of the successor function used by graph search?

Quick-check answers

1. The priority calculation changes; successor generation can remain identical.
2. Both methods need the same estimate of remaining grid distance, although A* also includes g .

3. The algorithm needs g to extend the path correctly; f alone mixes known and estimated cost.
4. Reading the current node's child references produces its successors.

Chapter recap

- The five search algorithms in Labs 1–3 share one generic search loop.
- Graphs, grids, words, and trees differ mainly in how successors are generated.
- Queue, stack/recursion, and heap data structures encode the frontier policy.
- Visited sets protect graph search from loops and repeated expansions.
- The templates make path, cost, and priority explicit.

Stage 2

Approximate Reasoning and Intelligent Control

Fuzzy Logic and ANFIS

Chapter at a glance

Before you start: Know basic true/false Boolean logic and be able to run MATLAB scripts.

By the end, you can: explain fuzzy truth values, identify the four components of a Fuzzy Inference System (FIS), distinguish Mamdani and Sugeno systems, explain the Adaptive Neuro-Fuzzy Inference System (ANFIS), and build/evaluate the MATLAB examples from Lab 4.

Lab 4 introduces **approximate reasoning**: fuzzy logic represents concepts such as “cold” or “hot” with degrees between 0 and 1.

4.1 From crisp truth to fuzzy membership

In classical logic, a statement is either false (0) or true (1). In fuzzy logic, a degree can lie anywhere in the interval

$$0 \leq \mu(x) \leq 1.$$

For example, “the water is hot” may have membership 0.7.

Conceptual intuition

Human descriptions often use graded language: somewhat cold, very fast, mostly full. Fuzzy logic turns that kind of gradual description into numerical membership values that a rule system can process.

Figure 4.1 shows the difference. A crisp set has a cliff: not hot at 29.9°, fully hot at 30.0°. A fuzzy set replaces it with a ramp, so a small change in temperature produces a small change in membership. That continuity is the practical advantage: it stops a controller juddering when a reading hovers near a threshold.

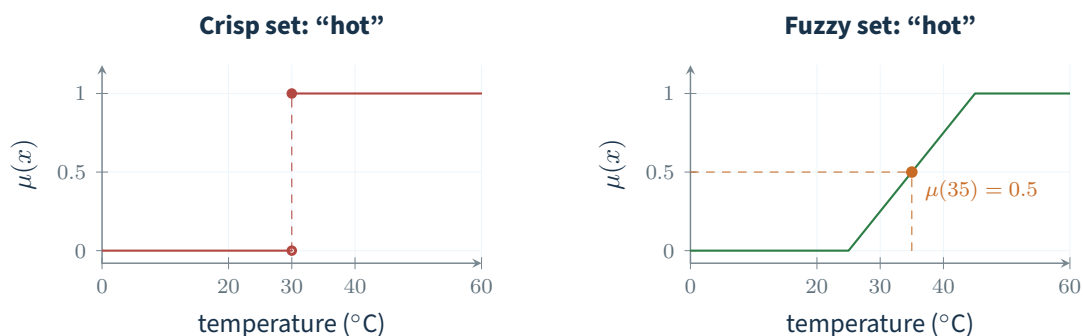


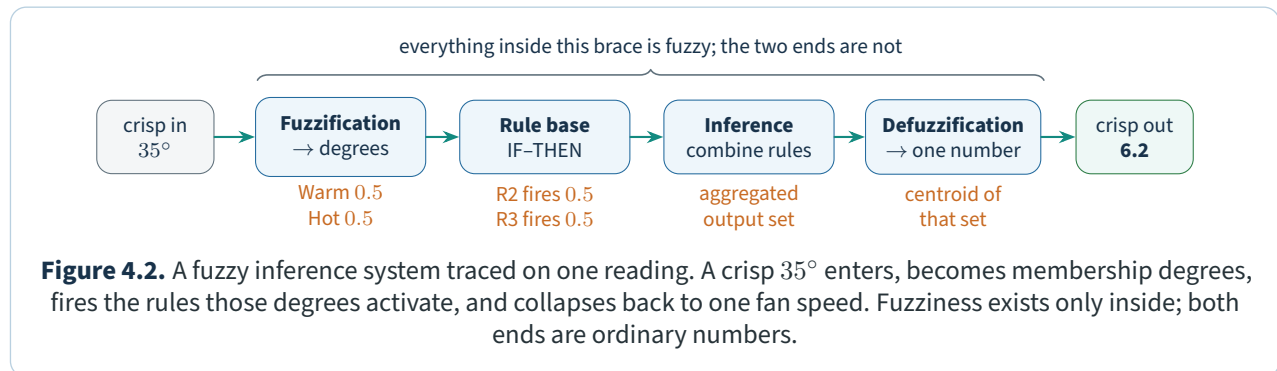
Figure 4.1. “Hot” as a crisp set and as a fuzzy set. The crisp version forces a decision at one point; the fuzzy version reports a degree, so 35° is hot to degree 0.5.

4.2 Why fuzzy logic is used

Fuzzy logic supports:

- handle uncertainty, imprecision, and approximate reasoning;
- support human-like linguistic rules;
- build decision and control systems for applications such as diagnosis and pattern recognition.

4.3 The four components of a Fuzzy Inference System



Component	Description from the laboratory material
Fuzzification	Converts crisp inputs to fuzzy values using membership functions.
Rule base	Stores IF-THEN rules, such as IF temperature is high THEN fan speed is high.
Inference engine	Applies the rules to the fuzzy inputs to obtain fuzzy outputs.
Defuzzification	Converts fuzzy outputs back to a crisp numerical value.

4.4 Membership functions in the temperature controller

The MATLAB example creates one input, Temperature, over $[0, 100]$ and gives it three triangular membership functions: Cold, Warm, and Hot. The output FanSpeed, over $[0, 10]$, receives Low, Medium, and High membership functions.

```

fis = mamfis('Name', 'temperatureControl');

fis = addInput(fis, [0 100], 'Name', 'Temperature');
fis = addMF(fis, 'Temperature', 'trimf', [0 0 50], ...
    'Name', 'Cold');
fis = addMF(fis, 'Temperature', 'trimf', [25 50 75], ...
    'Name', 'Warm');
fis = addMF(fis, 'Temperature', 'trimf', [50 100 100], ...
    'Name', 'Hot');

fis = addOutput(fis, [0 10], 'Name', 'FanSpeed');
fis = addMF(fis, 'FanSpeed', 'trimf', [0 0 5], ...
    'Name', 'Low');
fis = addMF(fis, 'FanSpeed', 'trimf', [3 5 7], ...
    'Name', 'Medium');
fis = addMF(fis, 'FanSpeed', 'trimf', [5 10 10], ...
    'Name', 'High');
  
```

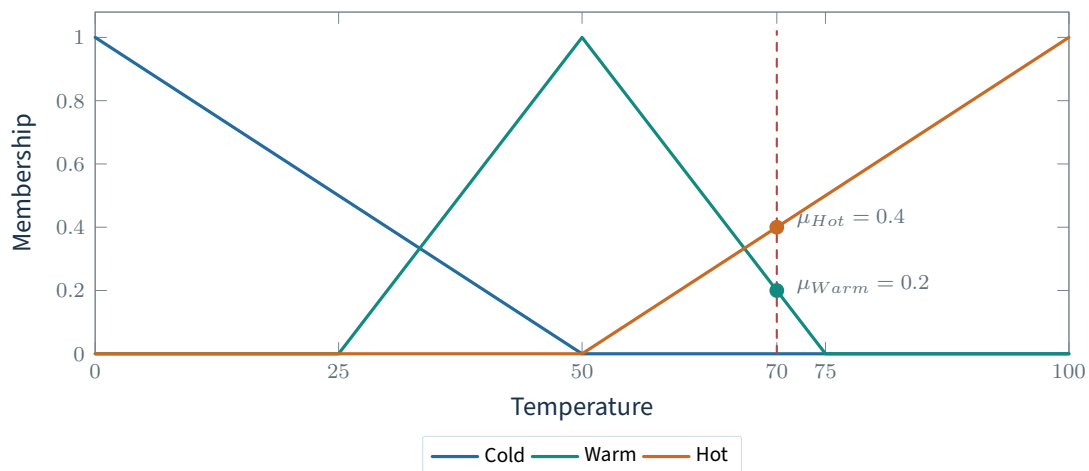


Figure 4.3. The three temperature membership functions. They deliberately overlap: at 70° the reading is simultaneously Warm to degree 0.2 and Hot to degree 0.4, so two rules fire at once and the controller blends their outputs instead of switching abruptly between them.

At a temperature of 70, overlapping memberships activate both Warm and Hot rules; fuzzy inference combines their contributions.

Combining them produces a fuzzy set, not a number, and a fan needs a number. Figure 4.4 shows the last stage: each firing rule clips its output set to the strength of its rule, the clipped shapes are merged, and the centroid of the merged area is reported as the answer.

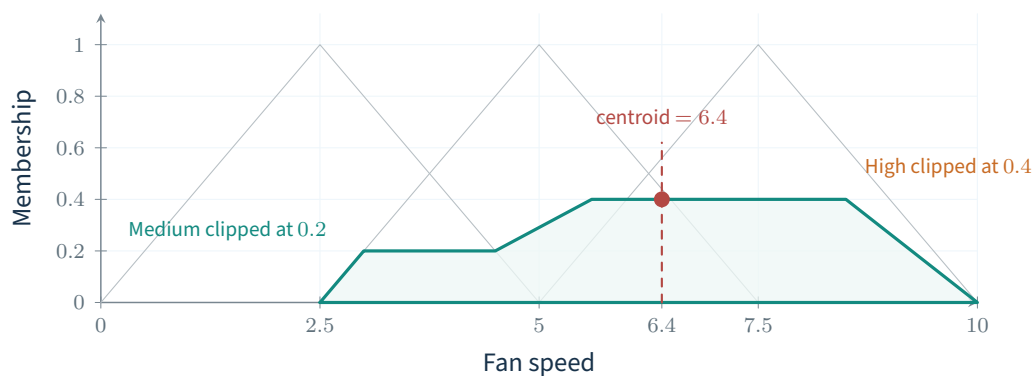


Figure 4.4. Defuzzifying the 70° reading. Medium is clipped at 0.2 and High at 0.4; the centre of area of their union gives the crisp fan speed. A stronger Hot rule pulls the centroid right.

The rule list maps each input term to its corresponding output term.

```
ruleList = [
    1 1 1 1; % IF Cold THEN Low
    2 2 1 1; % IF Warm THEN Medium
    3 3 1 1 % IF Hot THEN High
];
fis = addRule(fis, ruleList);

fanSpeed = evalfis(fis, 70);
fprintf('Fan speed at 70 C: %.2f\n', fanSpeed);
figure;
gensurf(fis);
```

Worked example

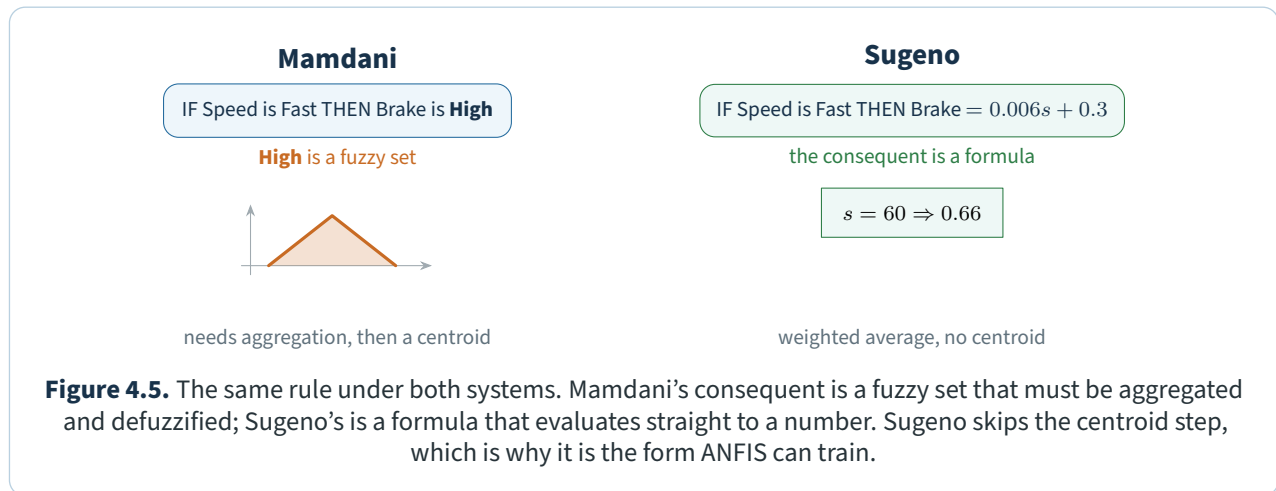
At 70°C, the triangular definitions give

$$\mu_{Cold}(70) = 0, \quad \mu_{Warm}(70) = \frac{75 - 70}{75 - 50} = 0.2, \quad \mu_{Hot}(70) = \frac{70 - 50}{100 - 50} = 0.4.$$

The Medium and High rules fire at strengths 0.2 and 0.4. Default Mamdani inference with centroid defuzzification produces a fan speed of approximately 7.16 on the [0, 10] scale.

4.5 Mamdani and Sugeno FIS

Lab 4 distinguishes two types of Fuzzy Inference System. They differ only in what a rule's *then* clause produces (Figure 4.5).



Type	Laboratory description
Mamdani	Intuitive and commonly used for control applications.
Sugeno	Uses constant or linear output functions and is suitable for optimization and learning.

The Sugeno example creates a Speed input and a BrakeForce output, with Gaussian input membership functions and linear output functions.

```

fis2 = sugfis('Name', 'speedControl');
fis2 = addInput(fis2, [0 100], 'Name', 'Speed');
fis2 = addOutput(fis2, [0 1], 'Name', 'BrakeForce');

fis2 = addMF(fis2, 'Speed', 'gaussmf', [10 20], 'Name', 'Slow');
fis2 = addMF(fis2, 'Speed', 'gaussmf', [10 80], 'Name', 'Fast');
fis2 = addMF(fis2, 'BrakeForce', 'linear', [0.002 0.1], 'Name', 'Low');
fis2 = addMF(fis2, 'BrakeForce', 'linear', [0.006 0.3], 'Name', 'High');

ruleList2 = [
    1 1 1 1; % IF Speed is Slow THEN BrakeForce is Low
    2 2 1 1 % IF Speed is Fast THEN BrakeForce is High
];
fis2 = addRule(fis2, ruleList2);

brakeForce = evalfis(fis2, 60);
fprintf('Brake force at speed 60: %.3f\n', brakeForce);

```

Each Sugeno rule produces a constant or linear value; the FIS returns their firing-strength-weighted average.

For a linear output MF with one input, MATLAB reads the parameters $[a \ b]$ as the function $a \times \text{Speed} + b$. At speed 60 the Gaussian memberships are $\mu_{\text{Slow}} = 0.0003$ and $\mu_{\text{Fast}} = 0.1353$, so the Fast rule dominates:

$$\text{BrakeForce} = \frac{0.0003 \times 0.22 + 0.1353 \times 0.66}{0.0003 + 0.1353} \approx 0.659.$$

Common mistake

Read the consequent function, not the rule's name. A rule labelled “High” whose function is $-0.01 \times \text{Speed} + 1$ still *decreases* with speed: it returns 1.0 at a standstill and 0.0 at 100. The label says high; the arithmetic says the opposite, and the arithmetic wins.

An earlier version of the supplied `Lab_4_code.m` used exactly those coefficients, giving

$$\text{BrakeForce}(20) = 0.200, \quad \text{BrakeForce}(60) = 0.400, \quad \text{BrakeForce}(100) = 0.000,$$

a controller that brakes hardest when parked and not at all at top speed. Every MATLAB call succeeded. The code was valid and the model was wrong.

The file now uses $[0.002 \ 0.1]$ and $[0.006 \ 0.3]$, so both consequents rise with speed and the numbers above hold.

Practice

Never judge a controller from one input value. Evaluate it across the whole range and look at the shape:

```
evalfis(fis2, [0; 20; 40; 60; 80; 100])
gensurf(fis2);
```

A monotonic rise from about 0.10 to 0.90 is what this design intends. Anything non-monotonic, flat, or falling means a consequent slopes the wrong way — the failure a single `evalfis(fis2, 60)` would have hidden completely.

4.6 ANFIS: Adaptive Neuro-Fuzzy Inference System

ANFIS can be summarized as

$$\text{ANFIS} = \text{neural-network learning} + \text{fuzzy-logic representation}.$$

It uses a Sugeno FIS and learns rule and membership-function parameters from data, commonly through least squares and gradient descent.

Figure 4.6 unpacks that sum. The five layers are the stages of Figure 4.2 redrawn as a network. Once each stage is a differentiable node, the membership shapes and rule outputs become parameters gradient descent can adjust. The fuzzy reasoning is unchanged — only who chooses the numbers.

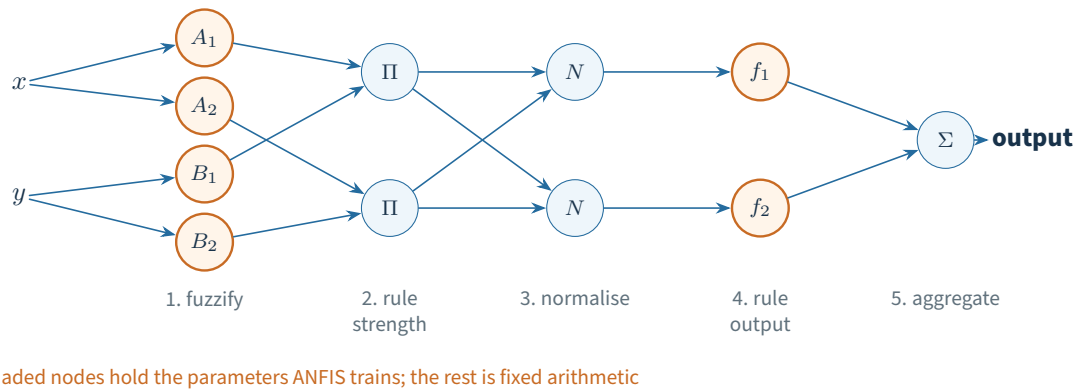


Figure 4.6. ANFIS for two inputs and two rules. Shaded layers (1 and 4) hold the trainable parameters; layers 2, 3, and 5 are fixed arithmetic. Same stages as Figure 4.2, drawn so a learning algorithm can reach inside.

Feature	Fuzzy Logic	ANFIS
Rule definition	Manual	Learned from data
Learning capability	No	Yes, through neural-network-style learning
Output type	Mamdani or Sugeno	Sugeno only
Typical use in handout	Expert-defined systems	Data-driven modeling

Key idea

Use manual fuzzy logic for expert-defined rules and ANFIS when data should determine model parameters.

4.7 Training examples in Lab 4

The examples cover one- and two-input training, prediction, FIS persistence, error analysis, and air-conditioning control.

```
% Single input
data1 = [0 0; 5 2.5; 10 5; 15 7.5; 20 10];
anfisFIS1 = anfis(data1, 10);

% Two inputs
data2 = [0 0 0; 5 2 2.5; 10 5 5; 15 7 7.5; 20 10 10];
anfisFIS2 = anfis(data2, 20);

% Prediction
inputVal = [12 6];
predictedOutput = evalfis(anfisFIS2, inputVal);
fprintf('Predicted output: %.3f\n', predictedOutput);
```

ANFIS interprets the final column as the target. Thus, data1 contains one input and one output, whereas data2 contains two inputs and one output. A prediction row contains inputs only: [12 6], without a placeholder for the unknown output.

Common mistake

Five rows demonstrate the application programming interface (API), not generalization. Evaluate separate validation data before making performance claims.

4.7.1 Scaling the diabetes data: two traps

The final ANFIS example trains on `diabetes.csv`, whose first eight columns are measurements and whose ninth (`Outcome`) is a 0/1 label.

Common mistake

`normalize(X)` **is not “scale to $[0, 1]$ ”**. With no options MATLAB performs *z-score* standardisation, giving mean 0 and standard deviation 1. Applied to the whole matrix it also rescales the label column, turning the 0/1 outcome into $\{-0.732, 1.365\}$ — no longer a label at all. The option that actually produces $[0, 1]$ is `normalize(X, 'range')`, and it belongs on the features only:

```
features = normalize(data3(:, 1:8), 'range'); % -> [0, 1]
labels   = data3(:, 9);                    % left as 0/1
```

Common mistake

A physically impossible zero is this dataset’s missing-value code. Insulin is 0 in 374 of 768 rows (49%) and SkinThickness in 227 (30%); BloodPressure, BMI and Glucose also contain zeros. Nobody has a blood pressure of zero.

Left as numbers they drag every mean and every membership function toward zero. Check before modelling:

```
sum(data3 == 0)
```

Then say in your report which columns you treated as missing and what you did about them.

Key idea

ANFIS regresses the 0/1 label, so its output is continuous. Turning that into a class needs a *threshold*, and 0.5 is a choice rather than a law. Compare the result against the majority-class baseline: 500 of 768 patients have Outcome 0, so always predicting 0 already scores about 65%. A model that cannot beat that has learned nothing worth reporting.

4.8 Software context

Lab 4 uses MATLAB’s Fuzzy Logic Toolbox, including `mamfis`, `sugfis`, `addMF`, `addRule`, `evalfis`, `gensurf`, `anfis`, `writefis`, and `readfis`.

PRACTICAL MILESTONE

Lab 4 of 8

Fuzzy Logic and ANFIS in MATLAB

Build, inspect, train, and save fuzzy systems

Coverage. Chapter 4 — fuzzy inference and ANFIS.

Source files. Lab04/Lab_4_code.m (all examples), ANFIS.m (the dataset part on its own), and diabetes.csv.

Goal. Build a Mamdani and a Sugeno system by hand, then let ANFIS learn the parameters from data instead. **Software.** MATLAB with Fuzzy Logic Toolbox.

Scaffolding

Run the examples in sequence so later models reuse the established objects and terminology.

Part A — Mamdani temperature controller.

Create temperatureControl, add Temperature and FanSpeed variables, define Cold/Warm/Hot and Low/Medium/High membership functions, add the three rules, evaluate at 70, and generate the FIS surface.

```
output = evalfis(fis, 70);
disp(['Fan Speed at 70 C: ', num2str(output)]);
gensurf(fis);
```

Part B — Sugeno controller and existing FIS.

Build speedControl with Gaussian input membership functions and linear output functions. Then load the source example tipper FIS and inspect its rules.

```
fis3 = readfis('tipper');
showrule(fis3);
```

Part C — ANFIS training and prediction.

Train the single-input and two-input examples, evaluate [12 6], save trainedFIS, and reload it.

```
writefis(anfisFIS2, 'trainedFIS');
loadedFIS = readfis('trainedFIS');
```

Part D — Data and error analysis.

Load diabetes.csv, normalize it, select columns 1, 2, and 9 for the first 500 rows, train ANFIS, and plot training error.

```
data3 = readmatrix('diabetes.csv');
data3 = normalize(data3);
dataSubset = data3(1:500, [1 2 9]);

anfisFIS3 = anfis(dataSubset, 2);
[anfisFIS4, trainError] = anfis(dataSubset, 2);
figure;
plot(trainError);
title('ANFIS Training Error');
xlabel('Epochs');
ylabel('Error');
```

Part E — Smart air conditioning (AC).

Use the source training table Temperature, Humidity → AC Intensity and evaluate the trained model at 37 degrees C and 65 percent humidity.

```
data4 = [20 30 0.3;
         25 40 0.5;
         30 50 0.7;
         35 60 0.9;
         40 70 1.0];
anfisFIS5 = anfis(data4, 2);
result = evalfis(anfisFIS5, [37 65]);
disp(['AC Intensity: ', num2str(result)]);
```

Practice

Change one membership function, compare the output at 70, and explain the change through rule activation.

Lab deliverable

Submit one MATLAB live script containing the Mamdani controller, Sugeno controller, ANFIS examples, one surface plot, one training-error plot, and a short comparison of manual FIS design versus learned ANFIS.

Check your understanding

1. What is the difference between a crisp value and a fuzzy membership value?
2. Name the four components of a Fuzzy Inference System in order.
3. Why does Lab 4 associate ANFIS with Sugeno rather than Mamdani output functions?
4. What does the smart air-conditioning ANFIS example use as inputs and output?

Quick-check answers

1. A crisp value is one measured number; a fuzzy membership is a degree in $[0, 1]$ describing how strongly that value belongs to a linguistic set.
2. Fuzzification, rule base, inference engine, and defuzzification.
3. ANFIS learns numerical rule consequents and parameters within a differentiable Sugeno structure.
4. Temperature and humidity are the inputs; air-conditioning intensity is the output.

Chapter recap

- Fuzzy logic represents degrees of truth between 0 and 1.
- A FIS consists of fuzzification, a rule base, an inference engine, and defuzzification.
- Mamdani systems aggregate fuzzy outputs; Sugeno systems use constant or linear consequents.
- ANFIS combines Sugeno fuzzy inference with data-driven learning.

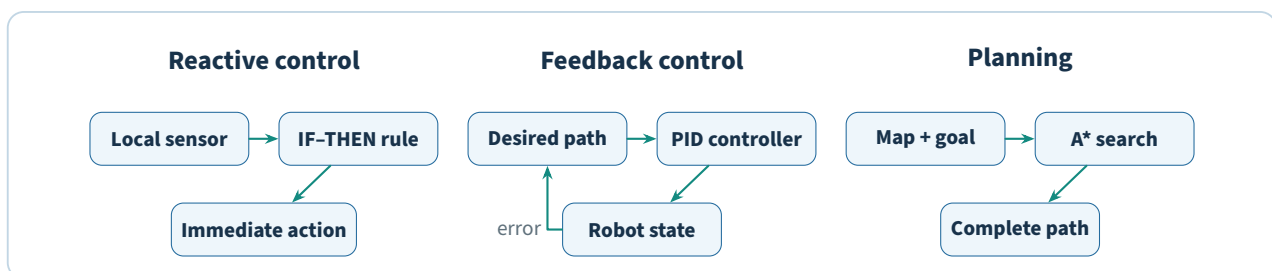
Robotics Scenarios and Simulation

Chapter at a glance

Before you start: Know basic Python, NumPy/Matplotlib, and the A* search idea from Chapter 2.

By the end, you can: explain the robot state updates used in Lab 5, simulate straight-line motion and obstacle avoidance, describe proportional–integral–derivative (PID) line following, apply A* to robot path planning, and interpret collision avoidance and 2-link forward kinematics.

Lab 5 applies AI methods to robot state, motion, feedback, and planning.



Reactive control selects an immediate action, feedback reduces observed error, and planning computes a sequence to the goal.

5.1 Scenario 1: simple two-dimensional (2D) robot motion

The first simulation starts at $(x, y) = (0, 0)$, sets the orientation to 45° , and moves with constant speed. The update rule in the source is

$$x \leftarrow x + v \cos(\theta) \Delta t, \quad y \leftarrow y + v \sin(\theta) \Delta t,$$

with $\Delta t = 0.1$.

```

import numpy as np
import matplotlib.pyplot as plt

x, y = 0, 0
theta = np.pi / 4
v = 1
time = np.linspace(0, 10, 100)
x_path, y_path = [], []

for t in time:
    x += v * np.cos(theta) * 0.1
    y += v * np.sin(theta) * 0.1
    x_path.append(x)
    y_path.append(y)

plt.plot(x_path, y_path, label='Robot Path')
plt.title('Simple 2D Robot Motion')
plt.xlabel('X')
plt.ylabel('Y')
plt.legend()
plt.grid(True)
plt.axis('equal')
plt.show()

print(f'Final position: ({x:.3f}, {y:.3f})')
```

Conceptual intuition

This baseline repeatedly updates position from speed and orientation; later controllers modify the resulting trajectory.

Worked example

At 45° , each update adds approximately (0.0707, 0.0707). After 100 updates, the position is (7.071, 7.071).

5.2 Scenario 2: obstacle avoidance by a reactive rule

The second scenario stores four discrete directions and checks whether the next cell is in a set of static obstacles. If an obstacle is detected, the direction index advances by one. With the usual Cartesian plotting convention, the sequence east, north, west, south produces a 90-degree counterclockwise turn.

```
import matplotlib.pyplot as plt

x, y = [0], [0]
directions = [(1, 0), (0, 1), (-1, 0), (0, -1)]
dir_index = 0
obstacles = {(3, 0), (4, 0), (0, 5)}

for _ in range(15):
    next_x = x[-1] + directions[dir_index][0]
    next_y = y[-1] + directions[dir_index][1]

    if (next_x, next_y) in obstacles:
        dir_index = (dir_index + 1) % 4
        continue

    x.append(next_x)
    y.append(next_y)

plt.scatter(*zip(*obstacles), marker='s', color='red', label='Obstacles')
plt.plot(x, y, 'o-', label='Robot path')
plt.axis('equal')
plt.grid(True)
plt.legend()
plt.show()
```

The robot reaches (2, 0), detects the blocked cell (3, 0), and turns north. The `continue` statement consumes that iteration without movement.

Figure 5.1 plots both scenarios. Scenario 1 has no sensing at all, so its path is a straight line whatever lies ahead. Scenario 2 adds one rule — turn if the next cell is blocked — and that is enough to route around an obstacle without any plan.

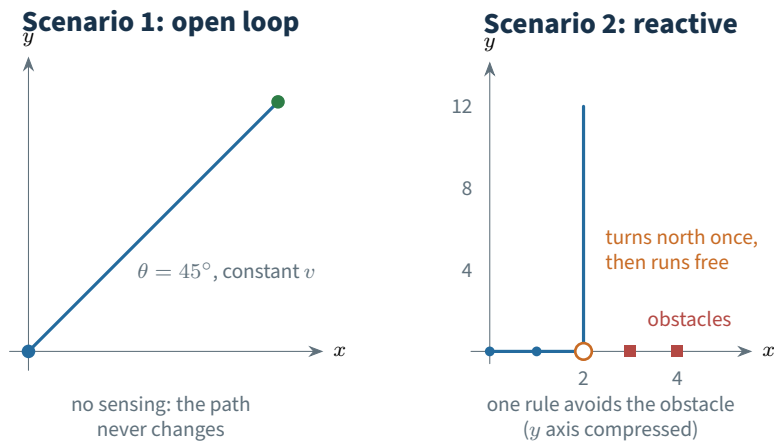


Figure 5.1. Scenario 1 (left) integrates position from a fixed heading and would drive straight through an obstacle. Scenario 2 (right) checks the next cell first: blocked at (3, 0), it turns north once and continues. Both paths are the exact output of the lab code.

Key idea

The controller responds to a blocked next cell without computing a complete route.

5.3 Scenario 3: line following with PID feedback

The line-following example defines a desired sinusoidal path

$$y_{\text{desired}} = 8 \sin(0.1x)$$

and updates the robot's vertical position using three error terms.

Term	Source variable	Role in the update
Proportional	$K_p * \text{error}$	Responds to the current difference between desired and actual path.
Integral	$K_i * \text{error_sum}$	Uses the accumulated error.
Derivative	$K_d * \text{derivative}$	Uses the change in error from the previous step.

The PID update is

$$y_{\text{robot}} \leftarrow y_{\text{robot}} + K_p e + K_i \sum e + K_d (e - e_{\text{prev}}).$$

```
import numpy as np
import matplotlib.pyplot as plt

Kp = 0.5
Ki = 0.01
Kd = 0.1

x_positions = np.linspace(0, 100, 300)
y_desired = 8 * np.sin(0.1 * x_positions)
y_robot = 0
error_sum = 0
prev_error = 0
y_path = []

for i in range(len(x_positions)):
    error = y_desired[i] - y_robot
    error_sum += error
```

```

    derivative = error - prev_error
    y_robot += Kp * error + Ki * error_sum + Kd * derivative
    prev_error = error
    y_path.append(y_robot)

plt.plot(x_positions, y_desired, '--', label='Desired path')
plt.plot(x_positions, y_path, label='Robot path')
plt.xlabel('x')
plt.ylabel('y')
plt.grid(True)
plt.legend()
plt.show()

```

Each PID iteration measures, accumulates, differentiates, and corrects the error before updating `prev_error`; updating it earlier would zero the derivative term.

Figure 5.2 shows what those three terms buy. Proportional control alone reacts to the current error and always trails the target, because the correction shrinks exactly as the error does. Adding the integral and derivative terms removes that lag: the integral keeps pushing while a steady offset persists, and the derivative damps the overshoot this would otherwise cause.

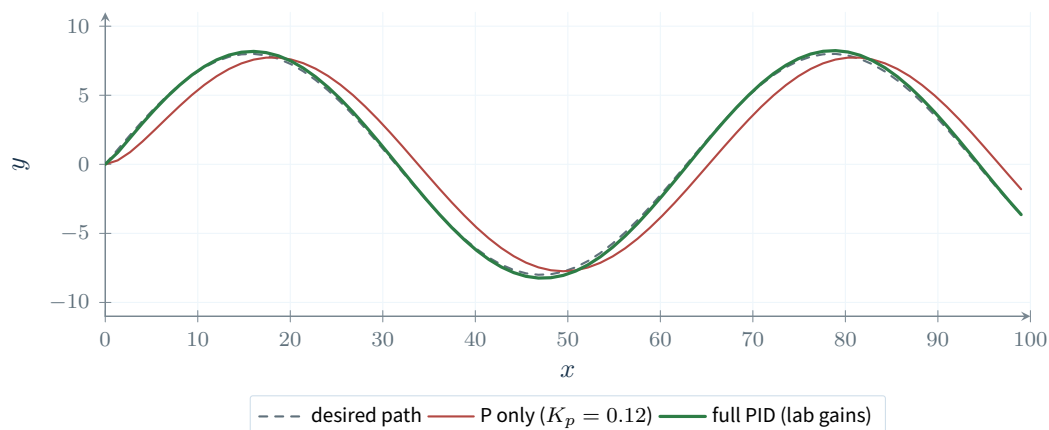


Figure 5.2. The line-following loop run twice with the same desired path. Proportional control alone lags behind the sine wave permanently; the full PID controller from the lab code tracks it closely. The gap between the two curves is what the integral and derivative terms are for.

5.4 Scenario 4: path planning with A*

The A* planner uses a priority queue, unit movement costs, and Manhattan distance.

```

import heapq

def heuristic(a, b):
    return abs(a[0] - b[0]) + abs(a[1] - b[1])

def a_star(grid, start, goal):
    open_list = []
    heapq.heappush(
        open_list,
        (heuristic(start, goal), 0, start, [])
    )
    visited = set()

    while open_list:
        _, cost, current, path = heapq.heappop(open_list)
        if current in visited:
            continue
        visited.add(current)
        path = path + [current]

```

```

    if current == goal:
        return path

    for dx, dy in [(0, 1), (1, 0), (-1, 0), (0, -1)]:
        nx, ny = current[0] + dx, current[1] + dy
        if 0 <= nx < len(grid) and 0 <= ny < len(grid[0]):
            if grid[nx][ny] == 0:
                heapq.heappush(
                    open_list,
                    (cost + 1 + heuristic((nx, ny), goal),
                     cost + 1,
                     (nx, ny),
                     path)
                )

    return []

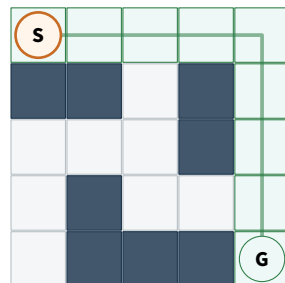
grid = [
    [0, 0, 0, 0, 0],
    [1, 1, 0, 1, 0],
    [0, 0, 0, 1, 0],
    [0, 1, 0, 0, 0],
    [0, 1, 1, 1, 0],
]
path = a_star(grid, (0, 0), (4, 4))
print(path)

```

Conceptual intuition

The search algorithm is the same idea as Chapter 2, but the state now has a physical interpretation: each node in the search graph is a possible robot location in the workspace grid.

Figure 5.3 shows the result on the lab's own grid. The planner expands 13 cells and returns a route of 8 moves, running along row 0 and then down column 4 — the only corridor the walls leave open.



Start (0,0), goal (4,4).
 13 cells expanded, 8 moves returned.
 Manhattan distance is admissible here
 because every move costs exactly 1.

Figure 5.3. The Scenario 4 grid with the route A* actually returns. Dark cells are obstacles, green cells are the returned path. The planner never sees a robot: to it these are just graph nodes, which is why the same code from Chapter 2 works unchanged.

5.5 Scenario 5: two robots avoiding collision

Two robots begin at opposite corners and move toward one another. The short source condition checks their current coordinate-wise separation. A complete simulation should check the *proposed next positions*; otherwise, two robots can move into the same cell between checks.

```

import numpy as np
import matplotlib.pyplot as plt

r1 = np.array([0, 0])
r2 = np.array([10, 10])
goal1 = np.array([10, 10])
goal2 = np.array([0, 0])
path1, path2 = [r1.copy()], [r2.copy()]

for _ in range(20):

```

```

step1 = np.sign(goal1 - r1).astype(int)
step2 = np.sign(goal2 - r2).astype(int)
next1, next2 = r1 + step1, r2 + step2

# Stop before the robots enter the same or adjacent cells.
if np.max(np.abs(next1 - next2)) <= 1:
    print('Potential collision detected; both robots stop.')
    break

r1, r2 = next1, next2
path1.append(r1.copy())
path2.append(r2.copy())

p1, p2 = np.array(path1), np.array(path2)
plt.plot(p1[:, 0], p1[:, 1], 'o-', label='Robot 1')
plt.plot(p2[:, 0], p2[:, 1], 's-', label='Robot 2')
plt.axis('equal')
plt.grid(True)
plt.legend()
plt.show()

```

Figure 5.4 plots the run. Both robots head straight for their goals along the diagonal, and the check fires one step before they would occupy the same cell.

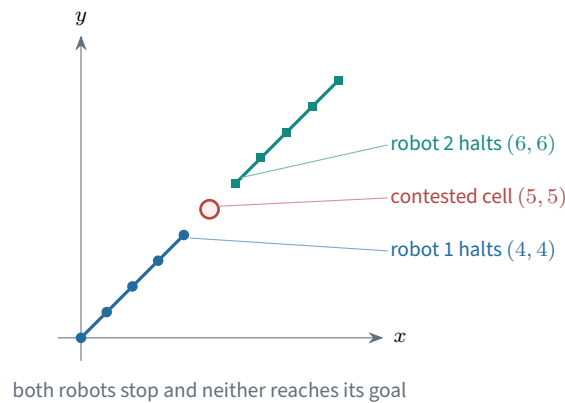


Figure 5.4. Scenario 5. The robots advance toward opposite corners until their proposed next positions would both be (5, 5); the rule halts them at (4, 4) and (6, 6). Note that both then stop permanently — detecting a collision is not the same as resolving one.

Common mistake

This stopping rule illustrates collision response but is not a complete multi-robot planner.

5.6 Scenario 6: a 2-link planar arm

For link lengths $l_1 = 5$, $l_2 = 3$ and joint angles θ_1, θ_2 , forward kinematics gives

$$\begin{aligned}
 x_{elbow} &= l_1 \cos \theta_1, & y_{elbow} &= l_1 \sin \theta_1, \\
 x_{tip} &= x_{elbow} + l_2 \cos(\theta_1 + \theta_2), & y_{tip} &= y_{elbow} + l_2 \sin(\theta_1 + \theta_2).
 \end{aligned}$$

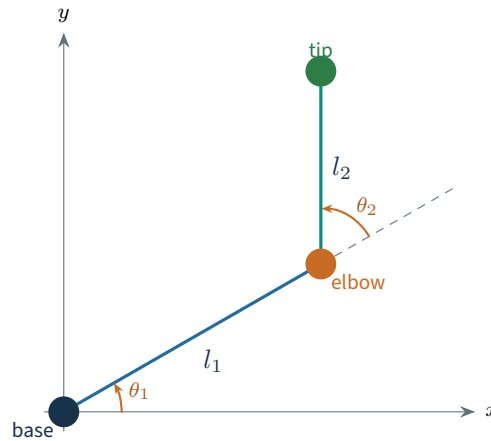


Figure 5.5. The 2-link planar arm. Joint angles are given as inputs and the tip position follows by trigonometry — forward kinematics. Going the other way, from a desired tip position to the joint angles, is the harder inverse problem and may have two solutions or none.

The first angle is measured from the global x -axis; the second is relative to link 1, giving absolute orientation $\theta_1 + \theta_2$.

```
import numpy as np
import matplotlib.pyplot as plt

l1, l2 = 5, 3
theta1_range = np.linspace(0, np.pi, 100)
theta2_range = np.linspace(0, np.pi/2, 100)

x2, y2 = [], []
for t1, t2 in zip(theta1_range, theta2_range):
    x_elbow = l1 * np.cos(t1)
    y_elbow = l1 * np.sin(t1)
    x_tip = x_elbow + l2 * np.cos(t1 + t2)
    y_tip = y_elbow + l2 * np.sin(t1 + t2)
    x2.append(x_tip)
    y2.append(y_tip)

plt.plot(x2, y2)
plt.xlabel('End-effector x')
plt.ylabel('End-effector y')
plt.title('2-link end-effector trajectory')
plt.axis('equal')
plt.grid(True)
plt.show()
```

Worked example

For $\theta_1 = 30^\circ$ and $\theta_2 = 60^\circ$, the elbow is $(5 \cos 30^\circ, 5 \sin 30^\circ) \approx (4.330, 2.500)$. The second link has absolute angle 90° , so it adds $(0, 3)$. The end effector is therefore approximately $(4.330, 5.500)$.

5.7 Additional scenarios named in the handout

The final page of the robotics source also outlines two later scenarios without supplying full programs.

Scenario 9: simultaneous localization and mapping (SLAM), static grid The robot moves and performs simulated scans in four directions. Grid value 1 denotes an obstacle and value 2 marks an obstacle that has been observed by the sensor.

Scenario 10: wall avoidance with Q-learning A Q-table stores action values and updates them from the immediate reward and best next-state value.

Second pass

The source omits Scenarios 7–8 and only outlines Scenarios 9–10. The complete versions below demonstrate map updates with known robot position and tabular Q-learning; neither is a full physical controller.

5.7.1 Complete static mapping extension

Unknown cells are -1 , observed free cells are 0, and observed obstacles are 2.

```
import numpy as np

world = np.array([
    [0, 0, 1, 0, 0],
    [0, 0, 1, 0, 1],
    [0, 0, 0, 0, 0],
    [1, 0, 1, 0, 0],
    [0, 0, 0, 0, 0],
])
route = [(4, 0), (4, 1), (3, 1),
         (2, 1), (2, 2), (2, 3)]
known = np.full(world.shape, -1)

for row, col in route:
    known[row, col] = 0
    for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
        nr, nc = row + dr, col + dc
        if 0 <= nr < world.shape[0] and 0 <= nc < world.shape[1]:
            known[nr, nc] = 2 if world[nr, nc] == 1 else 0

print(known)
```

5.7.2 Complete Q-learning wall-avoidance extension

For state s , action a , reward r , and next state s' , the update is

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right].$$

```
import numpy as np

grid = np.array([
    [0, 0, 0, 0, 0],
    [0, 1, 1, 1, 0],
    [0, 0, 0, 1, 0],
    [0, 1, 0, 0, 0],
    [0, 0, 0, 1, 0],
])
start, goal = (4, 0), (0, 4)
actions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
Q = np.zeros((*grid.shape, len(actions)))
rng = np.random.default_rng(7)
alpha, gamma, epsilon = 0.2, 0.95, 0.15

for episode in range(500):
    state = start
    for _ in range(100):
        if rng.random() < epsilon:
            action = rng.integers(len(actions))
        else:
            action = int(np.argmax(Q[state]))

        dr, dc = actions[action]
        candidate = (state[0] + dr, state[1] + dc)
        valid = (0 <= candidate[0] < grid.shape[0] and
                 0 <= candidate[1] < grid.shape[1] and
                 grid[candidate] == 0)
        next_state = candidate if valid else state
```



```
reward = 20 if next_state == goal else (-5 if not valid else -1)

old = Q[state + (action,)]
target = reward + gamma * np.max(Q[next_state])
Q[state + (action,)] = old + alpha * (target - old)
state = next_state
if state == goal:
    break

state, learned_path = start, [start]
for _ in range(30):
    if state == goal:
        break
    action = int(np.argmax(Q[state]))
    dr, dc = actions[action]
    state = (state[0] + dr, state[1] + dc)
    learned_path.append(state)
print(learned_path)
```

Step, wall, and goal rewards shape the policy. Verify that the stochastic result stays in bounds, avoids walls, and reaches the goal.

PRACTICAL MILESTONE

Lab 5 of 8

Robotics scenarios and simulation in Python

From motion equations to robot decision rules

Coverage. Chapter 5 — motion, reactive control, feedback control, and planning.

Source files. Lab05/Lab_5_code.ipynb.

Goal. Turn motion equations into reactive and feedback control rules, and compare planned motion with sensed motion. **Software.** Python, NumPy, Matplotlib, and heapq for A*.

Scaffolding

Use each plot to explain why the implemented trajectory has its observed shape.

Part A — Motion and reactive control.

Run Scenario 1 and Scenario 2. For Scenario 1, verify that a 45-degree heading produces equal-size x and y increments. For Scenario 2, plot obstacles and the robot path, then record where each 90-degree turn occurs.

Part B — Feedback control.

Run the PID line-following simulation with $K_p = 0.5$, $K_i = 0.01$, and $K_d = 0.1$. Plot the desired path and robot path together.

Part C — Deliberative planning.

Run the A* grid planner on the supplied 5-by-5 obstacle map. Render the path using S, G, *, #, and . exactly as in the handout.

Part D — Multi-robot and arm motion.

Run the two-robot collision-avoidance example and the 2-link arm example. For the arm, plot the end-effector trajectory.

Practice

Change one parameter in one scenario, predict the effect, and compare the prediction with the result.

Lab deliverable

Submit one notebook containing the six implemented scenarios, all resulting plots/text-grid outputs, and a one-paragraph explanation of how reactive control, feedback control, and search-based planning differ in the examples.

Check your understanding

1. In simple 2D motion, what do $\cos \theta$ and $\sin \theta$ control?
2. What event causes the obstacle-avoidance robot to change its direction index?
3. What three forms of error are used by the PID example?
4. What search method is reused for grid path planning?
5. What two equations determine the tip position of the 2-link arm?

Quick-check answers

1. They resolve the speed into horizontal and vertical displacement.
2. The next proposed grid cell is present in the obstacle set.
3. Present error, accumulated error, and change in error.
4. A* search.
5. $x = l_1 \cos \theta_1 + l_2 \cos(\theta_1 + \theta_2)$ and $y = l_1 \sin \theta_1 + l_2 \sin(\theta_1 + \theta_2)$.

Chapter recap

- Robot behavior is represented through state updates and trajectories.
- Straight-line motion uses velocity and orientation; obstacle avoidance uses a local condition-action rule.
- PID line following feeds current, accumulated, and changing error back into the motion update.
- A* becomes a robot path planner when grid coordinates are interpreted as physical positions.
- Collision avoidance and forward kinematics connect control logic to robot state.

Stage 3

Language Intelligence

Natural Language Processing Foundations

Chapter at a glance

Before you start: Be able to run MATLAB and read simple vector, image, and text-processing code.

By the end, you can: explain the purpose and significance of natural language processing, compute sentence similarity from word embeddings, describe the handout's classification-based image-caption simulation, and build an optical character recognition pipeline with dictionary-based translation.

Natural Language Processing (NLP) applies linguistics, computer science, and machine learning to analyze or generate text and speech.

6.1 Why NLP is needed

NLP converts unstructured language into representations suitable for interaction, automation, and information extraction.

Applications include:

- healthcare: analyzing patient notes and assisting diagnosis;
- business: chatbots, sentiment analysis, and automated email sorting;
- education: grammar correction and automated essay scoring;
- law: legal-document summarization and contract analysis;
- robotics: verbal commands and real-time interaction.

Key idea

Each example follows a pipeline: input, representation, and task-specific output.

6.2 Text similarity with word embeddings

The first MATLAB example compares

“The quick brown fox jumps over the lazy dog.”

with

“A fast brown animal leaps above a sleeping dog.”

using pretrained FastText word embeddings.

The pipeline has five stages.

Step 1: Tokenize both sentences.

Step 2: Load pretrained FastText embeddings.

Step 3: Remove words not available in the embedding vocabulary.

Step 4: Convert each word to a vector and average the word vectors for each sentence.

Step 5: Compute cosine similarity between the two sentence vectors.

```
clear; clc;

text1 = "The quick brown fox jumps over the lazy dog.";
text2 = "A fast brown animal leaps above a sleeping dog.";

documents = tokenizedDocument([text1, text2]);
emb = fastTextWordEmbedding;

words1 = string(documents(1).Vocabulary);
words2 = string(documents(2).Vocabulary);
words1 = words1(isVocabularyWord(emb, words1));
words2 = words2(isVocabularyWord(emb, words2));

if isempty(words1) || isempty(words2)
    error("A sentence has no words in the embedding vocabulary.");
end

vecs1 = word2vec(emb, words1);
vecs2 = word2vec(emb, words2);
vec1 = mean(vecs1, 1);
vec2 = mean(vecs2, 1);

similarity = dot(vec1, vec2) / (norm(vec1) * norm(vec2));
fprintf("Cosine Similarity between the two texts: %.4f\n", similarity);
```

The Vocabulary property contains unique word types, so repeated words do not receive extra weight in this version. A token-level sentence representation would preserve repetition. This distinction matters when word frequency carries meaning.

Common mistake

Averaging word vectors throws away word order. The mean of a set of vectors does not depend on the order they were added, so

“dog bites man” and “man bites dog”

produce *exactly* the same sentence vector and a cosine similarity of 1.000. The two sentences describe opposite events, and this representation cannot tell them apart at all.

Because the code averages over Vocabulary rather than tokens, it is also blind to repetition: “very very good” and “very good” are identical too.

Try it — it takes one line to change the input, and seeing 1.000 come back makes the limitation concrete in a way that reading about it does not.

Key idea

This is the **bag-of-embeddings** trade. Averaging is cheap, needs no training, and works surprisingly well for topic-level similarity — which is what the lab example measures. It is the wrong tool the moment meaning depends on *arrangement*: negation, subject versus object, or word order generally. Models that do handle order (recurrent networks, transformers) exist precisely because this limitation is fundamental rather than a detail of the implementation. Say which kind of similarity your result supports before quoting the number.

6.3 Cosine similarity

For sentence vectors $\mathbf{u}$ and $\mathbf{v}$, the code computes

$$\cos(\theta) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}.$$

A value near 1 indicates high directional similarity.

Conceptual intuition

A word embedding replaces a word with a numerical vector. Averaging the word vectors gives one rough vector for a sentence. Cosine similarity then asks whether the two sentence vectors point in similar directions.

Figure 6.1 shows why that averaging step is defensible. In a trained embedding space, words used in similar contexts end up close together, so the average of a sentence's word vectors lands in the region of meaning that the sentence is about. The space in the lab has hundreds of dimensions; the two drawn here stand in for that.

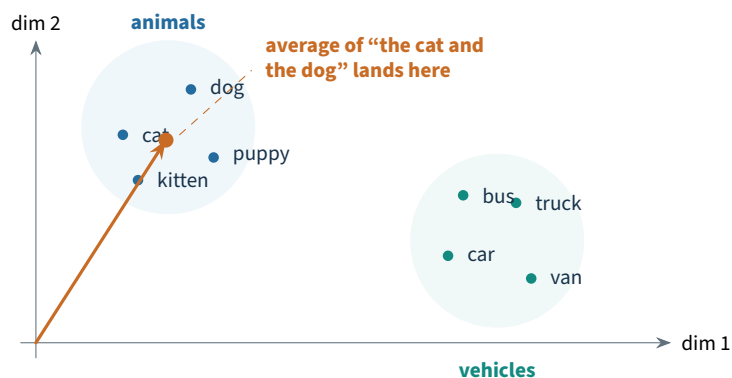


Figure 6.1. A two-dimensional sketch of an embedding space. Words that appear in similar contexts sit near one another, and a sentence vector — the average of its word vectors — lands inside the relevant cluster. This is what makes comparing two averaged sentences meaningful at all.

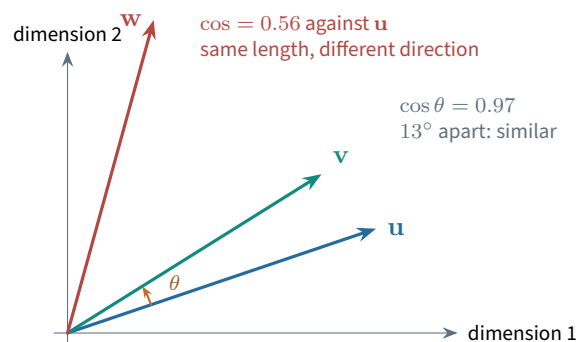


Figure 6.2. Cosine compares direction only. $\mathbf{u}$ and $\mathbf{v}$ sit 13° apart and score 0.97. $\mathbf{w}$ has almost the same length as $\mathbf{u}$ (3.58 against 3.59) yet scores 0.56 — length carries no weight.

Cosine similarity compares direction, not magnitude: opposite vectors score negative, perpendicular ones score zero, and length is ignored entirely. In Figure 6.2, $\mathbf{u}$ and $\mathbf{w}$ are almost the same length yet score only 0.56.

Worked example

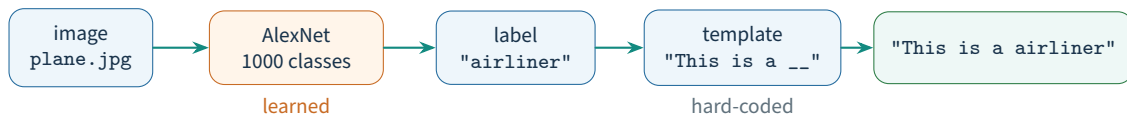
For a small numerical check, let $\mathbf{u} = (1, 1)$ and $\mathbf{v} = (2, 1)$. Their dot product is 3, their norms are $\sqrt{2}$ and $\sqrt{5}$, and their cosine similarity is

$$\frac{3}{\sqrt{2}\sqrt{5}} = \frac{3}{\sqrt{10}} \approx 0.949.$$

The value is high because the vectors point in similar directions; MATLAB applies the same calculation in higher dimensions.

6.4 Image captioning using AlexNet: a classification-based simulation

Figure 6.3 shows what the pipeline really does. AlexNet returns one class label; the “caption” is that label dropped into a fixed sentence template. Nothing in the system describes a scene — it names an object and wraps it in words.



one label per image means one possible sentence per label: the model cannot mention colour, count, or relationship

Figure 6.3. The Lab 6 captioning pipeline. A real captioning model generates a sentence conditioned on the image; this one substitutes a class label into a template, so it can only ever say one thing per label.

The second Lab 6 example loads an image, resizes it to AlexNet’s expected input size, extracts `fc7` features, classifies the image, and turns the predicted class label into a short caption.

```

clear; clc;
img = imread('plane.jpg');
imshow(img);
title('Input Image');

net = alexnet;
imgResized = imresize(img, net.Layers(1).InputSize(1:2));

featureLayer = 'fc7';
features = activations(net, imgResized, featureLayer, ...
    'OutputAs', 'rows');

[label, scores] = classify(net, imgResized);
disp(['Predicted Object: ', char(label)]);

caption = "This is a picture of a " + lower(char(label)) + ".";
disp("Generated Caption:");
disp(caption);
  
```

The `features` variable demonstrates how an intermediate neural representation can be extracted, but the fixed-template caption does not use it directly. The `classify` call produces the label inserted into the caption.

Common mistake

This is classification with a fixed caption template, not free-form image captioning.

6.5 Scene text detection and translation

The third Lab 6 pipeline uses optical character recognition (OCR) to extract printed text from an image and a hard-coded English-to-French dictionary to simulate translation without an internet application programming interface (API).

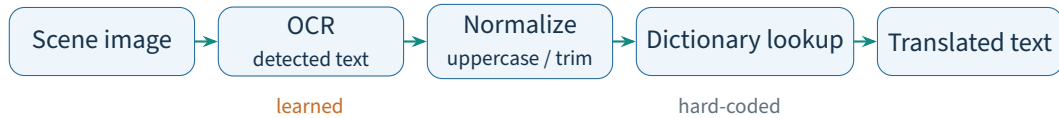


Figure 6.4. The scene-text pipeline. Only the OCR stage is a learned model; normalisation and lookup are ordinary string handling, which is why the whole thing runs without a network connection — and why an unseen word simply fails to translate.

```

img = imread('street.jpg');
imshow(img);
title("Scene Text Image");

ocrResult = ocr(img);
detectedText = upper(strtrim(ocrResult.Text));
disp("Detected Text:");
disp(detectedText);

dictionary = containers.Map;
dictionary("STOP") = "ARRET";
dictionary("SPEED LIMIT") = "LIMITATION DE VITESSE";
dictionary("YIELD") = "CEDEZ LE PASSAGE";
dictionary("NO PARKING") = "STATIONNEMENT INTERDIT";
dictionary("ONE WAY") = "SENS UNIQUE";
dictionary("DO NOT ENTER") = "ENTREE INTERDITE";
dictionary("STREET") = "RUE";

% Try longer phrases first so that a specific phrase wins.
sourcePhrases = string(keys(dictionary));
[~, order] = sort(strlength(sourcePhrases), 'descend');
sourcePhrases = sourcePhrases(order);
translatedText = "";

for i = 1:numel(sourcePhrases)
    phrase = char(sourcePhrases(i));
    if contains(detectedText, string(phrase))
        translatedText = string(dictionary(phrase));
        break;
    end
end

if translatedText == ""
    translatedText = "No dictionary translation found";
end
disp("French translation:");
disp(translatedText);
  
```

The contains check tolerates surrounding OCR text, but translation remains limited to stored dictionary phrases.

Worked example

"STOP\nAHEAD" still matches STOP and returns ARRET; S70P requires partial matching or a clearer image.

6.6 Extensions in the OCR examples

Lab 6 gives three additional variations.

Multi-language dictionary A STOP sign is mapped independently to French and German dictionaries.

Fuzzy keyword matching Partial keywords such as PARK or SPEED trigger translations even when the OCR text is not an exact dictionary key.

Region-of-interest (ROI) selection The user draws a rectangle around the relevant image region before OCR, reducing the text-recognition area.

```
img = imread('stop.jpg');  
imshow(img); title("Select Region for OCR");  
roi = drawrectangle;  
pause(1);  
ocrRes = ocr(img, roi.Position);  
text = upper(strtrim(ocrRes.Text));  
disp("Detected Text: " + text);
```

PRACTICAL MILESTONE

Lab 6 of 8

Natural Language Processing — Part A

Embeddings, image classification captions, and OCR translation

Coverage. Chapter 6 — text as data, OCR, embeddings, and captioning.

Source files. Lab06/TextSimilarityusingWordEmbeddings.m, ImageCaptioningusingAlexNet.m, OCRwithROI.m, SceneTextDetectionandTranslation.m, with the supplied .jpg images.

Goal. Represent text as vectors for similarity, and run an image-to-caption and an OCR-to-translation pipeline end to end. **Software.** MATLAB with the toolboxes/models required by the source examples.

Scaffolding

Run the three main pipelines and OCR variations separately before combining them.

Part A — Sentence similarity.

Run the two-sentence FastText example. Record the cosine-similarity output and list which tokens survive the embedding-vocabulary filter.

Part B — Classification-based caption.

Run the AlexNet example with plane.jpg. Display the input image, predicted object label, and generated fixed-template caption.

Part C — OCR and translation.

Run the street-sign OCR example with the English-to-French dictionary. Then test the multi-language STOP dictionary, partial keyword matching, and ROI-selection examples.

Practice

Add one road-sign phrase and compare exact matching with OCR text that contains surrounding words.

Lab deliverable

Submit one MATLAB live script containing all Lab 6 pipelines, the similarity value, the image classification/caption output, OCR text, at least two translated outputs, and a short note describing what each stage contributes.

Check your understanding

1. What role do pretrained FastText vectors play in the similarity example?
2. Why are the word vectors averaged before cosine similarity is computed?
3. What produces the caption text in the AlexNet example?
4. What is the difference between OCR and the dictionary translation step?
5. Why can selecting a region of interest change the OCR result?

Quick-check answers

1. FastText maps recognized tokens to semantic vectors learned from a large text corpus.
2. Averaging creates one fixed-length sentence representation from a variable number of word vectors.
3. A fixed sentence template inserts AlexNet's predicted class label.
4. OCR recognizes characters in pixels; the dictionary maps recognized phrases to stored translations.
5. A region of interest removes irrelevant pixels and can make the target text larger and less ambiguous to OCR.

Chapter recap

- NLP connects human-language data with computer representations and tasks.
- Lab 6 uses pretrained embeddings and cosine similarity for semantic comparison.
- The AlexNet example classifies an image and converts the class label into a simple caption template.
- OCR extracts printed text from images; a local dictionary then simulates translation.
- Dictionary design and ROI selection affect recognition and translation quality.

NLP Methods: Entities, Summaries, Sentiment, and Keywords

Chapter at a glance

Before you start: Understand the Lab 6 pipeline idea and basic MATLAB string operations.

By the end, you can: explain the nine Lab 7 examples, distinguish rule-based from feature-based NLP, interpret frequency and term frequency–inverse document frequency (TF–IDF) summarization, and identify limitations of simplified simulations.

Lab 7 surveys nine compact NLP pipelines based on rules, text statistics, or a simple classifier.

7.1 Code 1: Named Entity Recognition (NER) simulation

The program tokenizes a sentence and matches each token against person, location, and organization dictionaries.

```
clc; clear;
text = "Barack Obama was born in Hawaii and served as the 44th President of the United States.";

personList = ["Barack Obama"];
locationList = ["Hawaii", "United States"];
organizationList = ["United Nations", "Google", "Microsoft"];

tokens = split(text);
disp("Named Entities Identified:");

for i = 1:length(tokens)
    word = erase(tokens{i}, '.');
    if any(strcmp(word, personList))
        fprintf("Person: %s\n", word);
    elseif any(strcmp(word, locationList))
        fprintf("Location: %s\n", word);
    elseif any(strcmp(word, organizationList))
        fprintf("Organization: %s\n", word);
    end
end
```

Figure 7.1 shows the mismatch. Splitting on whitespace produces single tokens, but two of the three dictionaries hold two-word entries. Those entries can never match, so the loop finds only Hawaii.

dictionary holds "Barack Obama" as one entry:
no single token can match it

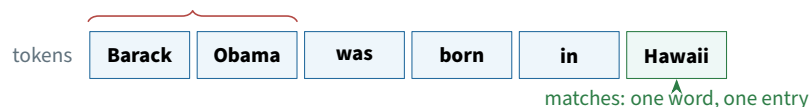


Figure 7.1. Tokenising on whitespace yields one word per token, while the dictionaries store multi-word names. Only the single-word entry Hawaii can ever match. The fix is to compare n -grams, not tokens — a representation problem, not a logic error.

Common mistake

The original code splits the sentence into single tokens but stores multi-word entities such as “Barack Obama” and “United States” as whole dictionary entries. Those entries cannot match a single token in the loop. Consequently, the original output contains only `Location: Hawaii`. This mismatch shows why representation choices matter.

A rule-based correction searches the normalized sentence for complete dictionary phrases:

```
cleanText = erasePunctuation(text);
entityGroups = {personList, locationList, organizationList};
groupNames = ["Person", "Location", "Organization"];

for g = 1:numel(entityGroups)
    entries = entityGroups{g};
    for i = 1:numel(entries)
        if contains(lower(cleanText), lower(entries(i)))
            fprintf('%s: %s\n', ...
                char(groupNames(g)), char(entries(i)));
        end
    end
end
```

This detects Barack Obama, Hawaii, and United States, but cannot identify entities absent from the dictionaries.

7.2 Code 2: summarization by keyword frequency

Before any counting can happen the text must be reduced to comparable units (Figure 7.2). Each stage throws information away deliberately: case, punctuation, common words, and finally word endings.

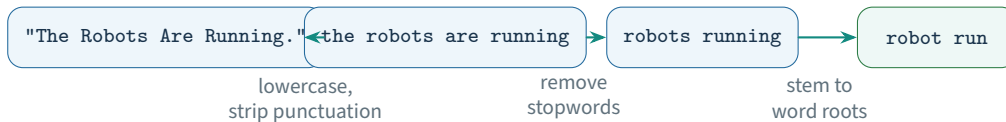
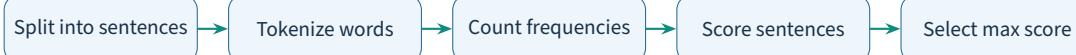


Figure 7.2. Preprocessing reduces raw text to countable tokens. Every stage discards something on purpose — which is also how it goes wrong: stemming “organisation” and “organ” to the same root makes them indistinguishable downstream.

The method counts words across the paragraph, scores each sentence by those frequencies, and selects the highest score.



The method is **extractive**: the summary is one of the original sentences rather than newly generated prose.

```
sentences = [
    "AI systems process language."
    "AI systems learn patterns from data."
    "Careful evaluation makes AI systems more reliable."
];

allWords = strings(0, 1);
for i = 1:numel(sentences)
    words = split(lower(erasePunctuation(sentences(i))));
    allWords = [allWords; words(words ~= "")];
end

vocabulary = unique(allWords);
```

```

frequency = zeros(size(vocabulary));
for j = 1:numel(vocabulary)
    frequency(j) = sum(allWords == vocabulary(j));
end

scores = zeros(numel(sentences), 1);
for i = 1:numel(sentences)
    words = split(lower(erasePunctuation(sentences(i))));
    for j = 1:numel(words)
        idx = find(vocabulary == words(j), 1);
        if ~isempty(idx)
            scores(i) = scores(i) + frequency(idx);
        end
    end
end

[~, best] = max(scores);
disp("Summary: " + sentences(best));
disp(table(sentences, scores));

```

Raw-frequency scoring may favor long sentences and function words; consider stop-word removal or length normalization.

7.3 Code 3: grammar correction simulation

The grammar-correction simulation handles one pattern through string replacement:

```

inputSentence = "She go to school every day.";
patterns = ["go to school"];
corrections = ["goes to school"];

corrected = inputSentence;
for i = 1:length(patterns)
    corrected = strrep(corrected, patterns(i), corrections(i));
end
disp("Original: " + inputSentence);
disp("Corrected: " + corrected);

```

The program applies a fixed replacement rule rather than inferring grammar.

7.4 Code 4: dependency-parsing simulation as part-of-speech (POS) tags

The example pairs each word in “The dog chased the cat.” with a manual tag. Here, DET means determiner, NOUN noun, and VERB verb:

DET, NOUN, VERB, DET, NOUN.

```

sentence = "The dog chased the cat.";
words = split(erasePunctuation(sentence));
tags = ["DET", "NOUN", "VERB", "DET", "NOUN"];

for i = 1:length(words)
    fprintf("%-7s - %s\n", words{i}, tags{i});
end

```

Conceptual intuition

The programmer supplies the tags, so the code demonstrates tagged-text representation rather than automatic parsing.

7.5 Code 5: simplified Naive Bayes sentiment classification

Codes 1–4 encode knowledge the programmer already has. Code 5 is the first that does not: it counts how often each word appeared in positive and negative training sentences, then scores a new sentence by multiplying those counts together (Figure 7.3). The “naive” assumption is that the words are independent given the label — which is plainly false for language, and works anyway.

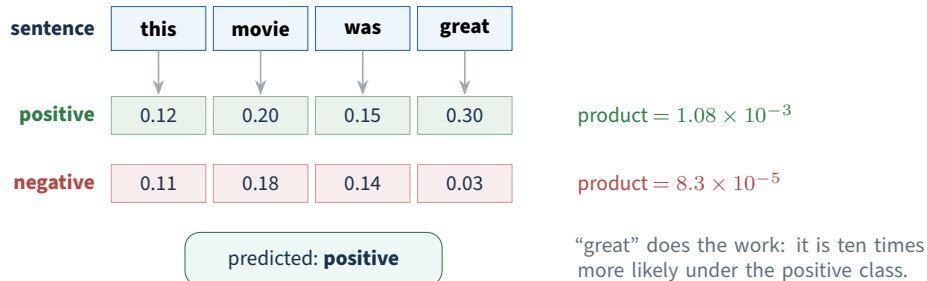


Figure 7.3. Naive Bayes on a two-class problem. Each word contributes an independent likelihood, the products are compared, and the larger one wins. Nothing here is a rule the programmer wrote: every number comes from counting the training data.

Five labeled sentences are converted to binary word features, used to train `fitcnb`, and tested on “The product is wonderful.”

```
texts = [
    "I love this product", ...
    "This is terrible", ...
    "Very happy with the results", ...
    "It is bad and disappointing", ...
    "Absolutely wonderful experience"
];
labels = [1; 0; 1; 0; 1];
bag = ["love", "terrible", "happy", "bad", "wonderful"];

features = zeros(length(texts), length(bag));
for i = 1:length(texts)
    for j = 1:length(bag)
        if contains(lower(texts(i)), bag(j))
            features(i, j) = 1;
        end
    end
end

keep = var(features) > 0;
features = features(:, keep);
activeBag = bag(keep);
Mdl = fitcnb(features, labels, 'DistributionNames', 'kernel');

testText = "The product is wonderful";
testFeatures = zeros(1, numel(activeBag));
for j = 1:numel(activeBag)
    testFeatures(j) = contains(lower(testText), activeBag(j));
end

prediction = predict(Mdl, testFeatures);
if prediction == 1
    disp("Predicted Sentiment: Positive");
else
    disp("Predicted Sentiment: Negative");
end
```

For the supplied test sentence, the program reports Predicted Sentiment: Positive.

The same `keep` mask must be applied to both training and test features. Otherwise, the classifier receives a different number or order of columns during prediction.

7.6 Code 6: summarization with TF-IDF

Raw frequency treats every word as equally informative, which is why “the” outranks the topic. TF-IDF fixes this by scaling each count down by how many documents the word appears in (Figure 7.4). A word that occurs everywhere carries almost no information about *which* document you are reading, and its weight collapses accordingly.

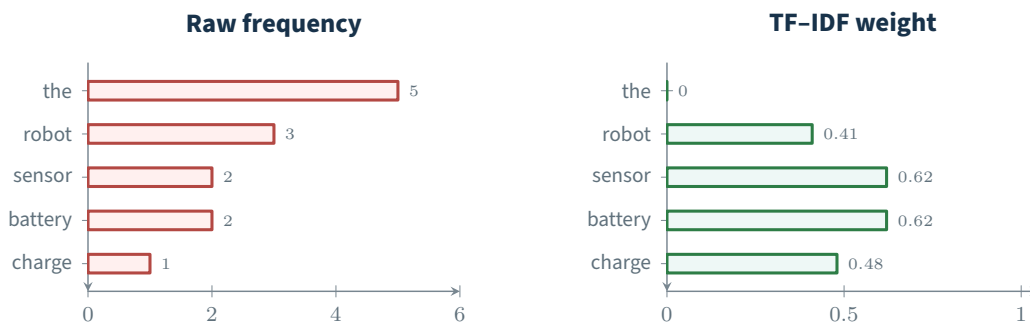


Figure 7.4. The same three-sentence document scored two ways. Raw frequency puts the stopword “the” on top; TF-IDF divides it out because it appears in every sentence, letting the words that actually distinguish the document rise. Nothing was removed by hand — the weighting did the filtering.

Lab 7 then uses MATLAB’s text-processing objects: `tokenizedDocument`, `bagOfWords`, and `tfidf`. Four sentences are converted into a TF-IDF matrix, sentence scores are computed by summing each row, and the maximum is returned.

```
text = [
    "Artificial intelligence is a broad field."
    "Machine learning enables systems to learn patterns from data."
    "NLP helps computers process human language."
    "Robotics connects sensing, planning, and action."
];

documents = tokenizedDocument(text);
bag = bagOfWords(documents);
tfidfMatrix = tfidf(bag);
score = sum(tfidfMatrix, 2);
[~, idx] = max(score);
disp("Summary: " + text(idx));
disp(table(text, score));
```

For the supplied text, the program selects “Machine learning enables systems to learn patterns from data.” Summed TF-IDF may favor sentences with more distinct terms; compare length-normalized scores when sentence lengths differ.

7.7 Code 7: rule-based sentiment analysis

The second sentiment method uses positive and negative word lists rather than a trained classifier.

```
text = "The scenery was amazing but the final scene was dull.";
positiveWords = ["good", "great", "excellent", "fantastic", ...
    "amazing", "breathtaking"];
negativeWords = ["bad", "terrible", "awful", "boring", ...
    "worst", "dull"];

words = lower(split(erasePunctuation(text)));
posCount = sum(ismember(words, positiveWords));
negCount = sum(ismember(words, negativeWords));

if posCount > negCount
    sentiment = "Positive";
```

```
elseif negCount > posCount
    sentiment = "Negative";
else
    sentiment = "Neutral";
end
fprintf('Positive=%d, Negative=%d, Result=%s\n', ...
    posCount, negCount, char(sentiment));
```

The program returns Positive, Negative, or Neutral according to which count is larger.

7.8 Code 8: heuristic language detection

The language detector defines small keyword lists for English, French, and German. It counts word matches for each list, takes the largest count, and maps the index to a language label. For the supplied French sentence, the program reports French.

```
text = "Bonjour, comment allez-vous aujourd'hui";
english = ["the", "and", "hello", "good"];
french = ["bonjour", "comment", "vous", "aujourd'hui"];
german = ["hallo", "wie", "geht", "ihnen"];

normalized = lower(erase(text, [",", ".", "!", "?"]));
normalized = replace(normalized, "-", " ");
words = split(normalized);
enCount = sum(ismember(words, english));
frCount = sum(ismember(words, french));
deCount = sum(ismember(words, german));

counts = [enCount, frCount, deCount];
languages = ["English", "French", "German"];
[bestCount, idx] = max(counts);
if bestCount == 0
    result = "Unknown";
elseif sum(counts == bestCount) > 1
    result = "Ambiguous";
else
    result = languages(idx);
end
disp("Detected language: " + result);
```

The zero-match and tie checks prevent the program from reporting English merely because it occupies the first array position.

7.9 Code 9: keyword extraction

The final example tokenizes a paragraph, removes stop words and punctuation, builds a bag-of-words model, sorts vocabulary counts in descending order, and returns the top five words.

```
text = "Data science involves extracting insights from structured " + ...
    "and unstructured data using statistical and machine " + ...
    "learning techniques.";

document = tokenizedDocument(text);
document = lower(document); % fold case BEFORE counting
document = erasePunctuation(document); % strip punctuation ...
document = removeStopWords(document); % ... then remove stop words

bow = bagOfWords(document);
[sortedCounts, sortedIndex] = sort(bow.Counts, 'descend');
sortedWords = bow.Vocabulary(sortedIndex);

n = min(5, numel(sortedWords));
disp(table(sortedWords(1:n)', sortedCounts(1:n)', ...
    'VariableNames', {'Keyword', 'Count'}));
```

Common mistake**Two preprocessing details decide whether this ranking means anything.**

Case. `tokenizedDocument` is case sensitive, so `Data` at the start of a sentence and `data` in the middle are different tokens. In the supplied paragraph “`data`” occurs three times but is counted as `Data` = 2 and `data` = 1 — the most important keyword in the text, understated by a third. Call `lower(document)` before counting.

Order. Strip punctuation *before* removing stop words. A token still carrying a full stop does not match the stop-word list, so `and .` survives a removal that `and` would not.

```
document = tokenizedDocument(text);
document = lower(document);           % fold case first
document = erasePunctuation(document);
document = removeStopWords(document); % then remove stop words
```

Common mistake

Print the counts, not only the words. After case folding, “`data`” appears 3 times and every other term appears once. A top-five list built from that is one real keyword followed by four arbitrary ties, resolved by whatever order the tokenizer happened to produce.

A ranking in which almost everything ties is not a ranking. Report the counts alongside the words so the reader can see how thin the evidence is, and do not present all five as findings.

7.10 A taxonomy of the Lab 7 methods

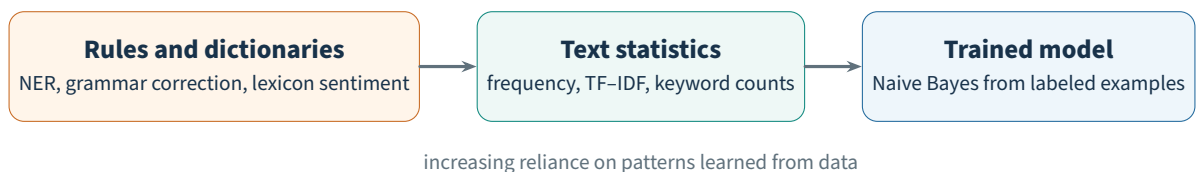


Figure 7.5. The nine Lab 7 programs arranged by the source of their decision logic. Moving right shifts the work from the programmer to the data. The axis is not a quality ranking: a dictionary lookup beats a trained classifier whenever the rules are genuinely known.

The spectrum identifies the source of decision logic: programmer rules, corpus statistics, or labeled training data. It is not a quality ranking.

Example	Main mechanism	What supplies the intelligence
NER simulation	Exact dictionary matching	Hand-written entity lists
Frequency summary	Word counts	Repetition-based sentence score
Grammar correction	String replacement	Hand-written correction rules
POS-style output	Fixed tag list	Programmer-supplied tags
Naive Bayes sentiment	Binary word features + classifier	Labeled examples and manual vocabulary
TF-IDF summary	Weighted term representation	Document statistics
Rule sentiment	Positive/negative lexicons	Hand-written sentiment lists
Language detection	Keyword counts	Language-specific keyword lists
Keyword extraction	Bag-of-words counts	Frequency after stop-word/punctuation removal

PRACTICAL MILESTONE

Lab 7 of 8

Natural Language Processing — Part B

Nine compact NLP pipelines in MATLAB

Coverage. Chapter 7 — tokenization, tagging, named entities, and sentiment.

Source files. Lab07/NER.m and CODE_2.m through CODE_9.m, one file per example.

Goal. Compare hand-written rules, statistical text representations, and a simple trained classifier.

Scaffolding

Reproduce each baseline result before extending the small examples.

Part A — Rule/dictionary pipelines.

Run Codes 1, 3, 4, 7, and 8. For each one, list the dictionary, pattern, tag list, or lexicon that the programmer supplied manually.

Part B — Summarization pipelines.

Run Code 2 and Code 6 on their supplied text. Record the selected sentence from each method and explain the scoring signal: raw frequency versus TF-IDF.

Part C — Learned sentiment classifier.

Run Code 5 and reproduce the source prediction for “The product is wonderful.” Print the binary feature row for the test sentence before prediction.

Part D — Keyword extraction.

Run Code 9 and record the top five keywords after stop-word and punctuation removal.

Practice

Modify the rule-based NER simulation to detect one multi-word entity and compare both outputs.

Lab deliverable

Submit one MATLAB live script containing Codes 1–9, the requested outputs, and a comparison table classifying each method as rule/dictionary based, statistics based, or trained classifier based.

Check your understanding

1. Why does the NER simulation miss multi-word entities under its current tokenization rule?
2. What makes the frequency and TF-IDF summarizers extractive?
3. Which sentiment example is trained from labeled data, and which uses a fixed lexicon?
4. What does the language detector do when it receives the keyword match counts?
5. Why are stop-word removal and punctuation removal useful before the keyword-count step in Code 9?

Quick-check answers

1. A phrase such as Barack Obama cannot equal either single token Barack or Obama.
2. They select and return an existing input sentence instead of generating a new sentence.
3. Code 5 trains Naive Bayes from labeled examples; Code 7 counts words in fixed sentiment lexicons.
4. It selects the language with the largest supported count; the completed version also handles zero matches and ties.
5. They prevent frequent grammatical words and punctuation marks from occupying the limited keyword list.

Chapter recap

- Lab 7 compares rules, text statistics, and a simple classifier.
- Frequency and TF-IDF can score existing sentences for extractive summarization.
- Sentiment can be implemented as a learned classification problem or as direct lexicon counting.
- Preprocessing choices expose limitations such as exact-match tokenization.

Stage 4

Evolutionary Optimization

Genetic Algorithms

Chapter at a glance

Before you start: Be comfortable with objective functions, random sampling, and MATLAB arrays.

By the end, you can: define the core Genetic Algorithm (GA) vocabulary, trace the GA workflow, explain selection/crossover/mutation, and apply the Lab 8 code patterns to continuous optimization, regression, knapsack, and polynomial-root objectives.

A **Genetic Algorithm (GA)** evolves a population of candidate solutions through selection, crossover, and mutation.

8.1 Core GA vocabulary

Concept	Description from the handout
Population	A set of possible solutions, also called individuals or chromosomes.
Chromosome	One complete candidate solution, represented by real values or a binary string in the examples.
Gene	One component or parameter inside a chromosome.
Fitness function	A function used to evaluate how good a candidate solution is.
Selection	Choose parents based on fitness, for example roulette-wheel selection.
Crossover	Combine two parents to produce offspring.
Mutation	Randomly alter genes to maintain diversity.
Elitism	Preserve the best solutions from one generation to the next.

8.2 The GA workflow

The workflow is:

Step 1: Initialize a population randomly.

Step 2: Evaluate the fitness of every individual.

Step 3: Select parents based on fitness.

Step 4: Apply crossover and mutation to create a new population.

Step 5: Replace the old population with the new generation.

Step 6: Repeat until convergence or the maximum generation count is reached.

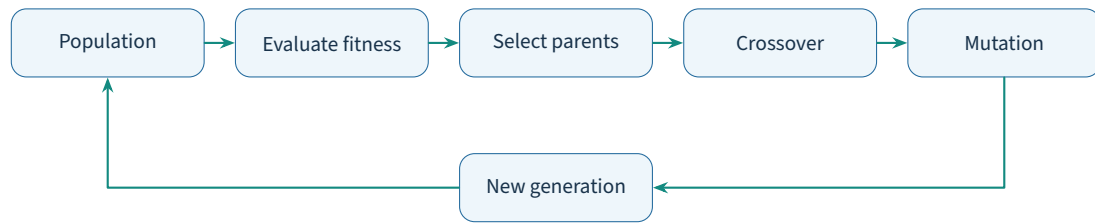


Figure 8.1. One generation of a genetic algorithm. Selection supplies the pressure toward better solutions, crossover recombines what already works, and mutation supplies the novelty that stops the population collapsing onto one answer.

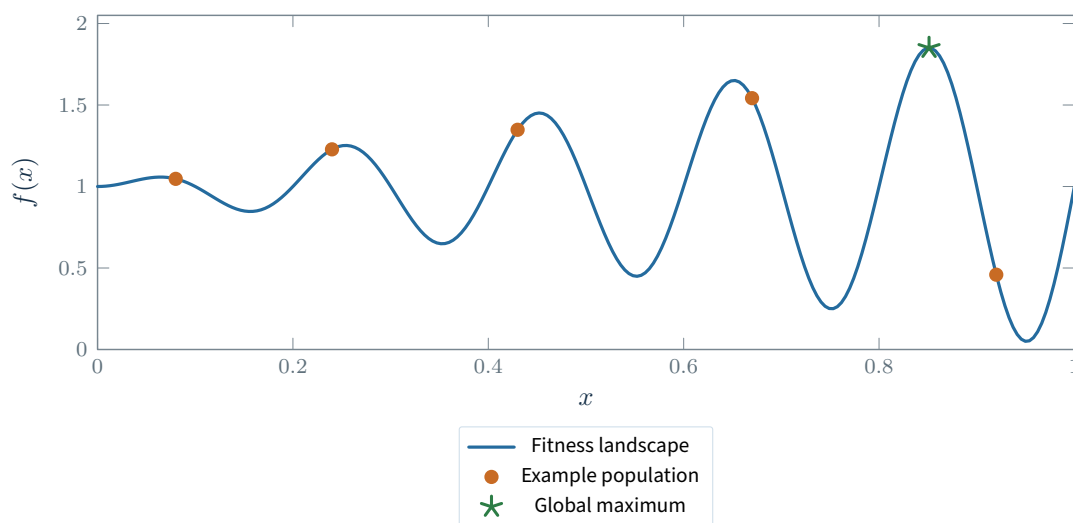
Key idea

A GA transforms a population: selection favors fitter candidates, crossover recombines them, and mutation adds variation.

8.3 Code 1: maximize a nonlinear function

The first problem maximizes

$$f(x) = x \sin(10\pi x) + 1, \quad x \in [0, 1].$$



The population samples several regions; selection favors high values while crossover and mutation continue exploration beyond local peaks.

The example uses 20 real-valued chromosomes, 50 generations, crossover rate 0.8, and mutation rate 0.1.

```

popSize = 20;
chromLength = 1;
maxGen = 50;
crossRate = 0.8;
mutRate = 0.1;

pop = rand(popSize, chromLength);
fitnessFunc = @(x) x .* sin(10 * pi * x) + 1;
  
```

8.3.1 Roulette-wheel selection

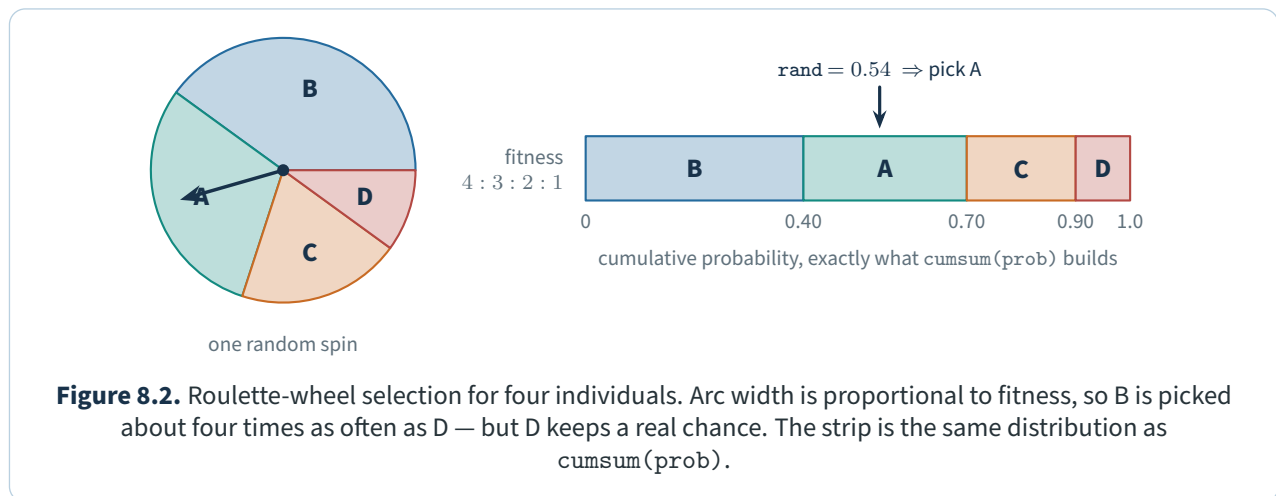
Fitness values define selection probabilities:

$$p_i = \frac{f_i}{\sum_j f_j},$$

then uses cumulative probabilities and a uniform random number to select each parent.

This formula requires nonnegative selection scores with a positive total. If an objective can be negative or if every score is zero, transform or shift the scores before normalizing them.

Figure 8.2 shows the mechanism. Each individual gets an arc proportional to its fitness and one random draw picks the owner. Fitter individuals win more often, but weak ones are never eliminated — keeping them preserves the diversity a later crossover may need.



```
fitness = fitnessFunc(pop);
prob = fitness / sum(fitness);
cumProb = cumsum(prob);
newPop = zeros(size(pop));

for i = 1:popSize
    r = rand;
    idx = find(cumProb >= r, 1, 'first');
    newPop(i,:) = pop(idx,:);
end
```

8.3.2 Arithmetic crossover and mutation

The continuous examples blend two parents with a random coefficient α , then occasionally add a small random perturbation.

Figure 8.3 contrasts the two continuous operators with the binary single-point crossover used for the knapsack problem in Code 4. Arithmetic crossover produces a child *between* its parents, so a population can only ever contract toward the region it already occupies; mutation is what lets it reach outside.

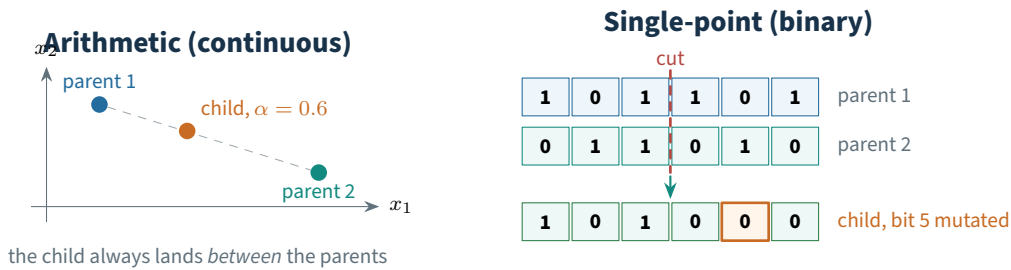


Figure 8.3. Left: arithmetic crossover places the child on the segment between its parents, so it cannot reach new territory alone. Right: single-point crossover swaps tails at a cut, and mutation flips one bit — the only operator that introduces a value neither parent had.

```
alpha = rand;
temp1 = alpha * newPop(i,:) + (1 - alpha) * newPop(i+1,:);
temp2 = alpha * newPop(i+1,:) + (1 - alpha) * newPop(i,:);
```

8.3.3 Complete runnable loop

The complete program combines initialization, evaluation, variation, replacement, reporting, and plotting.

```
rng(7); % reproducible teaching run
popSize = 20;
maxGen = 50;
crossRate = 0.8;
mutRate = 0.1;
pop = rand(popSize, 1);
fitnessFunc = @(x) x .* sin(10*pi*x) + 1;

bestHistory = zeros(maxGen, 1);
bestEverFitness = -Inf;
bestEverX = NaN;

for gen = 1:maxGen
    fitness = fitnessFunc(pop);
    [bestHistory(gen), bestIndex] = max(fitness);
    if bestHistory(gen) > bestEverFitness
        bestEverFitness = bestHistory(gen);
        bestEverX = pop(bestIndex);
    end

    probability = fitness / sum(fitness);
    cumulative = cumsum(probability);
    parents = zeros(size(pop));
    for i = 1:popSize
        index = find(cumulative >= rand, 1, 'first');
        parents(i) = pop(index);
    end

    offspring = parents;
    for i = 1:2:popSize
        if rand < crossRate
            alpha = rand;
            p1 = parents(i);
            p2 = parents(i+1);
            offspring(i) = alpha*p1 + (1-alpha)*p2;
            offspring(i+1) = alpha*p2 + (1-alpha)*p1;
        end
    end

    mutate = rand(size(offspring)) < mutRate;
    offspring(mutate) = offspring(mutate) + ...
        0.05*randn(sum(mutate(:)), 1);
    pop = min(max(offspring, 0), 1);
end

fprintf('Best x: %.5f, f(x): %.5f\n', ...
```

```

    bestEverX, bestEverFitness);
plot(bestHistory, 'LineWidth', 1.5);
xlabel('Generation'); ylabel('Best fitness in generation');
grid on;

```

The global maximum is near $x = 0.85119$, where $f(x) \approx 1.85060$; a short stochastic run may not reproduce it exactly.

Worked example

If three candidates have fitness $[1, 2, 1]$, their roulette probabilities are $[0.25, 0.50, 0.25]$, and their cumulative probabilities are $[0.25, 0.75, 1.00]$. A random draw $r = 0.60$ selects the second candidate because 0.75 is the first cumulative value at least as large as 0.60.

Core material

Codes 2–5 retain the loop while changing initialization, fitness conversion, bounds, and mutation.

Figure 8.4 shows one run of this configuration. Two things stand out. The best-so-far curve improves in jumps — generations 1–3, then 14, 27, and 30–31 — with flat stretches between. And the best individual *present* in each generation keeps falling below that line: the supplied code has no elitism, so a good chromosome that is not selected simply disappears.

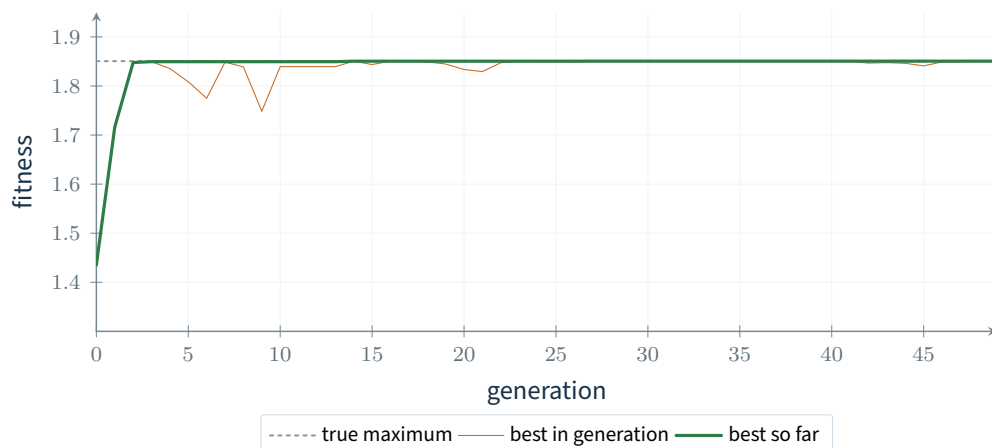


Figure 8.4. A run of Code 1 on $f(x) = x \sin(10\pi x) + 1$. Best-so-far can only rise, and does so in jumps separated by flat stretches. The best individual *present* in each generation keeps dipping below it: without elitism the algorithm loses its best chromosome and has to rediscover it. Axis cropped to $[1.3, 1.95]$.

8.4 Code 2: minimize the Rosenbrock function

The second problem minimizes

$$f(x, y) = (1 - x)^2 + 100(y - x^2)^2, \quad x, y \in [-2, 2].$$

Because roulette-wheel selection in the example expects larger values to be more desirable, the source inverts the minimization fitness:

$$\text{invFitness} = \frac{1}{f + 10^{-6}}.$$

```

pop = -2 + 4*rand(popSize, 2); % both genes in [-2, 2]
fitnessFunc = @(v) (1 - v(:,1)).^2 + ...
    100*(v(:,2) - v(:,1).^2).^2;

fitness = fitnessFunc(pop);

```

```

invFitness = 1 ./ (fitness + 1e-6);
prob = invFitness / sum(invFitness);

mutate = rand(size(newPop)) < mutRate;
newPop(mutate) = newPop(mutate) + ...
    0.05*randn(sum(mutate(:)), 1);
newPop = min(max(newPop, -2), 2);

```

The mutation step adds Gaussian noise and clips each variable back to $[-2, 2]$. The known optimum is $(x, y) = (1, 1)$, where the objective is zero; this gives a direct correctness check for the final chromosome.

8.5 Code 3: optimize linear-regression weights

The regression example generates noisy data from

$$y = 2.5x + 5 + \text{noise}$$

and represents each chromosome as two real-valued genes, w_1 and w_2 . The objective is mean squared error (MSE):

$$\text{MSE}(w_1, w_2) = \text{mean} (y - (w_1x + w_2))^2.$$

```

rng(7);
x = linspace(0, 10, 100)';
true_w = [2.5, 5];
y = true_w(1)*x + true_w(2) + randn(size(x));

pop = [5*rand(popSize, 1), 10*rand(popSize, 1)];
mse = zeros(popSize, 1);
for i = 1:popSize
    prediction = pop(i,1)*x + pop(i,2);
    mse(i) = mean((y - prediction).^2);
end
selectionScore = 1 ./ (mse + 1e-6);

```

A chromosome may encode model parameters directly. Evaluate $\text{pop}(i, 1)$ and $\text{pop}(i, 2)$ from the same row to avoid shape errors.

8.6 Code 4: 0–1 knapsack

The knapsack example uses binary chromosomes. Four genes indicate whether each item is selected. Figure 8.5 shows the encoding and the penalty that keeps infeasible solutions out of the gene pool.

	item 1	item 2	item 3	item 4	
	$w = 2, v = 3$	$w = 3, v = 4$	$w = 4, v = 5$	$w = 5, v = 8$	
feasible	1	1	0	0	weight $5 \leq 5$, value 7
infeasible	0	1	1	0	weight $7 > 5$, fitness set to 0

Figure 8.5. Each gene is one item: 1 means packed. The chromosome $[1, 1, 0, 0]$ weighs 5 and fits exactly; $[0, 1, 1, 0]$ weighs 7 and is over capacity, so its fitness is forced to zero and roulette selection never picks it.

```

weights = [2 3 4 5];
values = [3 4 5 8];
capacity = 5;

chromLength = length(weights);
pop = round(rand(popSize, chromLength));

fitness = zeros(popSize, 1);
for i = 1:popSize

```

```

totalWeight = sum(pop(i,:) .* weights);
totalValue = sum(pop(i,:) .* values);
if totalWeight <= capacity
    fitness(i) = totalValue;
end
end

```

Feasible chromosomes receive total value as fitness; overweight chromosomes receive zero. Mutation flips one bit:

```

gene = randi(chromLength);
newPop(i, gene) = 1 - newPop(i, gene);

```

Worked example

Chromosome $[1, 1, 0, 0]$ selects items 1 and 2, giving weight $2 + 3 = 5$ and value $3 + 4 = 7$. Chromosome $[0, 0, 0, 1]$ selects item 4, giving weight 5 and value 8. The latter is the optimum for this four-item instance. Chromosome $[1, 0, 1, 0]$ has weight 6 and therefore receives fitness zero under the stated penalty rule.

Conceptual intuition

Continuous optimization mutates a number by adding a small perturbation. Binary optimization mutates a decision by flipping 0 to 1 or 1 to 0. The GA framework is the same; the chromosome representation determines the appropriate operator.

8.7 Code 5: find roots through minimization

The final problem minimizes

$$f(x) = (x - 3)^2(x + 2)^2, \quad x \in [-5, 5].$$

The objective is zero at $x = 3$ and $x = -2$. Negating and shifting it produces nonnegative roulette-wheel selection scores.

```

pop = -5 + 10*rand(popSize, 1);
fitnessFunc = @(x) (x - 3).^2 .* (x + 2).^2;
fitness = -fitnessFunc(pop);

prob = fitness - min(fitness) + eps;
prob = prob / sum(prob);

```

Because both -2 and 3 produce objective zero, separate runs may converge to different correct roots. Report the final objective as well as x : a value near a root with a visibly nonzero objective remains an imprecise solution.

8.8 What changes across the five GA problems?

Problem	Chromosome	Fitness direction	Mutation
Nonlinear $f(x)$	One real value	Maximize	Small real perturbation
Rosenbrock	Two real values	Minimize	Gaussian perturbation, clipped to bounds
Linear regression	(w_1, w_2)	Minimize MSE	Gaussian perturbation
Knapsack	Four bits	Maximize feasible value	Flip one bit
Polynomial roots	One real value	Minimize objective	Gaussian perturbation, clipped to bounds

8.9 Elitism in the concept list versus the supplied code

Elitism copies the current best solution unchanged into the next generation. The supplied loops omit this step, so generation-best fitness need not be monotonic.

Debugging task

Add explicit elitism to one real-coded example: before replacement, identify the current best chromosome and force it into one position in `newPop`. Compare the best-fitness trace with the original script.

PRACTICAL MILESTONE

Lab 8 of 8

Genetic Algorithm optimization

Five GA formulations in MATLAB

Coverage. Chapter 8 — five GA formulations.

Source files. Lab08/CODE_1.m through CODE_5.m, one file per problem.

Goal. Keep the GA loop recognizable while changing chromosome representation and fitness definition to match each problem.

Scaffolding

Run the five baseline loops first; stochastic initialization and operators may produce different results. Each revised script reports the best final chromosome in problem-specific terms, not only a convergence curve.

Common mistake

Without elitism, later variation can remove the best chromosome. Distinguish the best fitness *seen* from the best individual that *survives* in the final population.

Part A — Continuous one- and two-variable objectives.

Run Code 1 and Code 2. Plot best fitness versus generation and record the best chromosome found at the end of the run.

Part B — Linear regression.

Run Code 3 and report the best discovered slope/intercept together with the final MSE. Compare the values qualitatively with the data-generating parameters [2.5, 5].

Part C — Binary knapsack.

Run Code 4. Decode the best chromosome into selected items, total weight, and total value. Confirm that the capacity constraint is satisfied.

Because only $2^4 = 16$ selections exist, compare the GA result with the brute-force optimum.

Key idea

An over-capacity chromosome receives fitness 0: a **penalty**, not a repair. Repairing or rejecting it would produce different search behavior.

Part D — Polynomial roots.

Run Code 5 and report whether the final population converges nearer to -2 or 3 in that run. Repeat with a different random seed and compare.

Practice

Add elitism to one code by copying the current best chromosome unchanged into the offspring. Compare the original and elitist traces and final solutions.

Troubleshooting

Minimization. Roulette selection requires larger scores for better candidates. Codes 2 and 5 transform the objective during selection:

- Code 2 keeps the raw Rosenbrock value, then selects on $\text{invFitness} = 1 ./ (\text{fitness} + 1e-6)$, so a smaller objective gives a larger share of the wheel.
- Code 5 negates the objective, then shifts with $\text{prob} = \text{fitness} - \min(\text{fitness}) + \text{eps}$ so the worst individual sits at zero probability rather than a negative one.

Both transformations assign larger selection scores to better candidates.

Run-to-run variation. GAs are stochastic; report results across several runs.

Different roots in Code 5. Both -2 and 3 are global minima with $f = 0$.

Suboptimal knapsack result. Increase population or generations, or add elitism.

Verification checklist

- Each script reports its decoded solution and convergence plot.
- The knapsack chromosome is decoded into items, weight, and value, and the capacity constraint is checked.
- Code 3's parameters are compared with $[2.5, 5]$.
- Code 5 is run more than once, with the root reported for each run.
- Differences between peak and final fitness are explained.

Lab deliverable

Submit one MATLAB live script containing all five GA problems, five convergence plots, the final decoded solution for each problem, and a comparison table listing chromosome type, fitness direction, crossover type, and mutation type.

Check your understanding

1. What is the difference between a chromosome and a gene in the Lab 8 terminology?
2. Why does the Rosenbrock example invert fitness values before roulette-wheel selection?
3. Why is bit-flip mutation appropriate for the knapsack example but not the regression-weight example?
4. What are the two exact roots encoded by the final polynomial objective?
5. What GA concept is defined in the handout but not explicitly implemented in the five source loops?

Quick-check answers

1. A chromosome is one complete candidate; a gene is one component of that candidate.
2. The raw objective is smaller for better solutions, whereas roulette selection assigns more probability to larger scores.
3. Knapsack genes are binary decisions, whereas regression genes are continuous numerical parameters.
4. $x = -2$ and $x = 3$.
5. Elitism.

Chapter recap

- A GA evolves a population through fitness evaluation, selection, crossover, mutation, and replacement.
- Chromosome representation determines how crossover and mutation should operate.
- The examples cover continuous, binary, regression, and root-finding problems.
- Roulette-wheel selection converts fitness into sampling probabilities.
- Elitism preserves the best solution but is absent from the supplied loops.

Quick Reference

Chapter at a glance

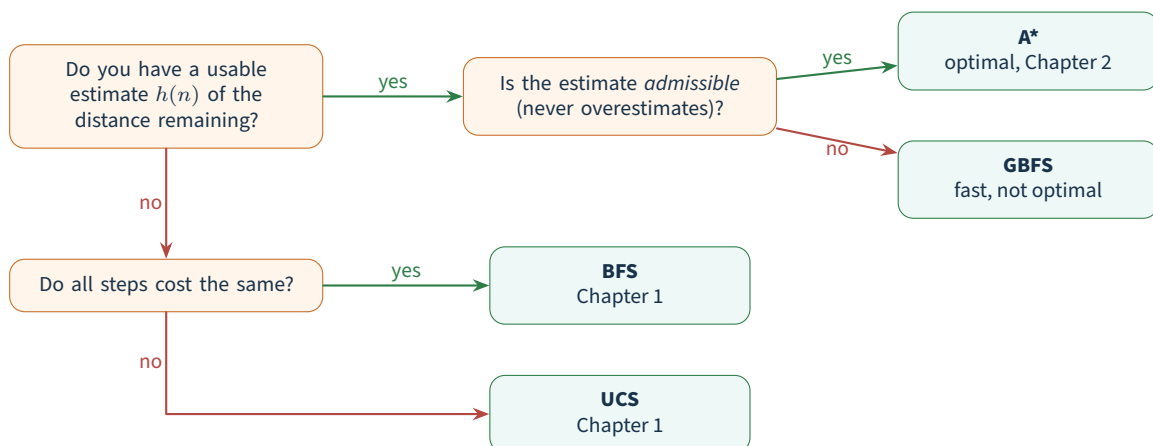
Before you start: Everything here has appeared earlier with full explanation; nothing in this appendix is new material.

By the end, you can: revise before an assessment, pick a method for a new problem, and check a property claim quickly.

This appendix gathers the decisions and numbers spread across the previous chapters. Use it for revision rather than first learning: every entry points back to the chapter that explains it.

A.1 Choosing a search algorithm

The five search algorithms in Chapters 1–3 differ in one respect only — what the frontier is sorted on — so the choice reduces to a short chain of questions about the problem (Figure A.1).



DFS does not appear on this path because it is never the choice when a *good* path is wanted. Use it when memory is the binding constraint, or when any reachable solution will do.

Figure A.1. A decision path for the five search algorithms in this course. Each question is about the *problem*, not the code: whether the steps cost the same, and whether a usable estimate of remaining distance exists.

A.2 Search algorithm properties

Method	Sorts on	Complete?	Optimal?	Space	Use when
BFS	depth	yes, finite b	only if equal costs	$O(b^d)$	fewest steps wanted
DFS	recency	no, in general	no	$O(bm)$	memory is tight
UCS	$g(n)$	yes, costs $\geq \varepsilon$	yes, non-negative costs	large	weighted, no heuristic
GBFS	$h(n)$	no	no	large	speed matters more than cost
A*	$g + h$	yes	yes, if h admissible	large	weighted, heuristic available

Key idea

Read the space column first. BFS, UCS, GBFS, and A* all store the whole frontier; on a large problem it is memory, not time, that stops them. DFS is the only one that does not, and it pays for that with both completeness and optimality.

A.3 Fuzzy logic, Mamdani, Sugeno, and ANFIS

	Mamdani	Sugeno	ANFIS
Output form	fuzzy set, then defuzzified	constant or linear function	Sugeno only
Rules from	an expert	an expert	learned from data
MF shapes from	an expert	an expert	learned from data
Best for	interpretable control	optimisation, computation	data-driven modelling
Needs data?	no	no	yes, and enough of it

A.4 The Lab 7 NLP methods by mechanism

Mechanism	Examples	Fails when
Rules and dictionaries	NER, grammar correction, lexicon sentiment, language detection	the input contains anything not in the list
Text statistics	frequency summarisation, TF-IDF, keyword extraction	the corpus is too small for the counts to mean anything
Trained model	Naive Bayes sentiment	the training data does not resemble the input

A.5 Genetic algorithm settings

Setting	Typical effect of raising it	Symptom that it is wrong
Population size	better coverage, slower generations	premature convergence if too small
Mutation rate	more exploration, less stability	fitness oscillates and never settles
Crossover rate	faster mixing of good material	diversity collapses if selection is also strong
Generations	more refinement	best-so-far curve flat for many generations
Elitism	guarantees the best is never lost	population converges on one answer early

A.6 Mistakes that recur across the labs

Mistake	What to do instead
Marking BFS states visited on removal	Mark on insertion, or the same state enters the queue repeatedly
Assuming a heuristic is admissible	Compare $h(n)$ against the true $h^*(n)$ before trusting A* (Chapter 2)
Expecting BFS to return the cheapest path	BFS counts edges, not cost (Figure 1.4)
Normalising fitness that can be negative	Shift or transform scores before dividing (Figure 8.2)
Updating <code>prev_error</code> too early in PID	The derivative term becomes zero (Chapter 5)
Judging a controller from one input	Sweep the whole range and check the shape (Chapter 4)
Scaling train and test data separately	Fit the scaler on training data only, then apply it